



面向 AI 智能体的操作系统内核

AgentOS-uCore: 基于 RISC-V uCore 的系统原生智能体运行机制

项目名称	面向 AI 智能体的操作系统内核
队伍名称	happy-legend
指导老师	余盛季、郭力维
目标平台	RISC-V 64 / uCore / QEMU
文档版本	决赛设计开发文档

姓名	年级	联系方式 (QQ)
王浩洋	大二	2091576055
康峻豪	大二	488718235

2026 年 08 月

摘要

智能体工作流已经从单轮问答发展为持续的工具调用、文件处理和多 Agent 协作。现有应用层框架能够安排模型和工具，却往往只在应用日志里保存调用者配置、工作流代次、上下文因果关系和资源归属。当工具真正写入文件、发送消息或申请 CPU 服务时，内核看到的仍是普通进程、文件和 IPC，无法据此作出一致判断。

本项目名称为“面向 AI 智能体的操作系统内核”，基于 LearningOS/uCore 实现 AgentOS-uCore。系统沿用 RISC-V uCore 的进程、虚拟内存、VFS、系统调用和调度器，并把 Agent（智能体）登记为内核能够识别的工作负载。Agent 进程创建与地址空间设计把身份、配额和生命周期状态留在 PCB，将高频 Context 镜像映射到固定用户地址；Agent-OS 内核结构化交互接口以 33 项工具目录、schema（参数模式）、执行约定（Execution Contract）和动态任务通道承接工具请求；Context Artifact Store 保存完整正文，内核封存 handle、producer、Task、来源序号和内容哈希；文件查询、事件等待、工作流 EEVDF 与有界查询预测分别处理属性检索、Agent Loop 唤醒、CPU 服务分配和低优先级预取。

在集成层，AgentOS 控制台按固定步骤执行科研恢复工作流；AgentOS Nexus 提供基于共同 Agent Loop 的通用 Harness。整个 Harness 会话共用一个长期运行 Guest，每个 Host Agent 都对应一个由 runtime SPAWN 创建的 Guest 进程，root Task 与动态子 Task 进入原生 Task Channel。每个 Agent 由 capability、允许工具、资源额度、系统提示和任务描述配置，名称只用于日志；拥有编排权限的 Agent 可以创建受限子 Agent，并以 128 字节 Task descriptor 绑定目标与输入 Artifact、权限、工具、workspace revision、预算、截止时间和结果类型。模型可以搜索与读取版本化工作区，也可以在固定路径内写入源码、生成补丁、隔离构建用户程序，并为每个用例启动独立 Guest。private Context 与 workflow 共享索引分别保存个人路径和已经结算的公共事实，重复只读查询还可训练固定容量转移表并触发受控预取。

性能测量采用 16 次相互独立的全新赛题平台启动，设置 $K = 4$ 次发现查询，A/B 与 B/A 顺序各 8 组。核心窗口中，遍历查询的中位耗时为 **121.206 ms**，索引复用路径为 **19.500 ms**，中位耗时之比为 **6.216 倍**；16/16 个配对均由索引复用路径更快完成，配对中位差为 **-105.668 ms**。完整流程的中位耗时由 **786.684 ms** 降至 **697.807 ms**，中位耗时之比为 **1.127 倍**，同样 16/16 个配对更快，配对中位差为 **-89.782 ms**。96 条元数据记录由 6 次批量调用登记，Catalog 冷建 1 次并复用 4 次，使检查记录数从 385 降至 5（77 倍），两条路径的结果哈希保持一致。文件查询网格覆盖目录规模与命中数的 12 种组合，任务通道保留 96 条包含 16 个操作的序列；EEVDF 的 504 条唤醒探针均在 0–1 个时钟节拍（tick）内获得服务。

表 1 将上述系统能力与运行时可观察行为对应起来。

设计主题	内核机制	可观察行为
Agent 进程创建与地址空间设计	PCB 扩展、受信任映像、七页 Agent Context 区	普通进程与 Agent 进程共存；Guest 可直接读取六页内核镜像，并使用一页用户缓存。
Agent-OS 内核结构化交互接口	工具目录、V2/V3 请求和固定大小返回结构	工具可发现，参数按 schema 解码；未知工具、类型错误和权限不足均返回明确状态。
工具调用协议	Execution Contract、Batch、任务通道 SQ/CQ、动态跨 Agent 委派	同步短调用、带约定调用和有界队列共用一套工具语义；child Task 通过 claim/complete、Artifact 和至多一条 terminal CQE 结算。
上下文路径管理	128 条有界路径、Context Artifact、private/shared 索引、分支和回滚	active path 保存可信短记录，完整正文可分页回查；父 Agent 只接纳成功结算的结果。
面向 Agent 查询优化的文件系统扩展	Metadata Catalog、Live Query、Typed Watch 和查询转移表	Agent 按属性选择文件，结构化历史在达到阈值后触发低优先级预取。
Agent Loop 内核运行机制	心跳、事件等待、消息路由、工作流 EEVDF	无事件时线程休眠，消息或 TIMER 到达后唤醒；多个工作流按实体而非线程数获得服务。
综合场景	恢复控制台和 Nexus 通用多 Agent Harness	完成动态 Task、长正文、版本化读写、隔离构建、独立 Guest 测试、摘要与会话关闭。

表 1: AgentOS-uCore 的系统能力与可观察行为

目录

摘要	ii
第一章 项目背景	1
1.1 行业背景与问题定义	1
1.1.1 从内容生成到可执行副作用	1
1.1.2 智能体执行的基础对象与风险链条	2
1.1.3 现有方案的特性与适用范围	3
1.1.4 AgentOS-uCore 的系统方案	4
1.2 核心挑战与设计目标	5
第二章 系统设计	7
2.1 AgentOS-uCore 总体架构	7
2.1.1 以 uCore 为底座的产品分层	7
2.1.2 六项核心机制与请求流向	8
2.1.3 一次智能体工具调用	9
2.1.4 上层 runtime	9
2.2 跨模块状态约束	11
第三章 系统实现	12
3.1 Agent 进程创建与地址空间设计	12
3.1.1 从普通进程到 workflow 中的 Agent	12
3.1.2 身份字段与角色策略	13
3.1.3 Agent Context 区的七页布局	13
3.1.4 生命周期代次与并发控制	13
3.1.5 随 workflow 创建的资源主体	14
3.1.6 验证结果	14
3.2 Agent-OS 内核结构化交互接口	14
3.2.1 工具调用协议与工具目录	14
3.2.2 自适应的 V2 调用	15
3.2.3 四种执行路径	15
3.2.4 工具执行期间的资源使用租约	16
3.2.5 带类型信息的任务通道	16
3.2.6 原生 delegated Task Channel	16
3.2.7 验证结果	17
3.3 上下文路径管理、数据来源与 Workflow Fence	17
3.3.1 从多轮调用到 Context Path	17
3.3.2 分支、回滚与有界淘汰	18
3.3.3 Context Artifact Store 与多 Agent 共享	18
3.3.4 结构化查询历史与预测输入	18
3.3.5 数据来源传播	19
3.3.6 工具清单与副作用检查	19
3.3.7 执行记录环	19
3.3.8 Workflow Fence	19
3.3.9 验证结果	20
3.4 面向 Agent 查询优化的文件系统扩展	20
3.4.1 从路径访问到属性查询	20
3.4.2 元数据登记与 inode 代次	20
3.4.3 遍历查询与选择性索引	21
3.4.4 带类型信息的实时订阅与快速跳过	21
3.4.5 从逐项建档到自适应复用	21
3.4.6 查询性能	22
3.4.7 验证结果	23
3.5 Agent Loop 内核运行机制	24
3.5.1 从轮询循环到内核等待	24
3.5.2 心跳与模型请求关联号	24
3.5.3 工作流资源账户	24
3.5.4 按工作流汇总的 EEVDF	25
3.5.5 唤醒延迟与公平性	25
3.5.6 验证结果	26
3.6 AgentOS 控制台综合工作流	26
3.6.1 科研恢复场景	26
3.6.2 遍历与索引路径的配对实验	26
3.6.3 核心查询与完整流程窗口	27
3.6.4 系统结果	28
3.7 综合场景：AgentOS Nexus 通用多 Agent Harness	29
3.7.1 共同 Agent Loop 与动态配置	29

3.7.2	一次交互回合	29
3.7.3	动态 delegated Task Channel	30
3.7.4	版本化工作区与受控 Broker	31
3.7.5	受控开发与真实 Guest 验证	31
3.7.6	Context Artifact Store 与长任务摘要	32
3.7.7	查询预测与受控预取	32
3.7.8	工具发现与内核观测	33
3.7.9	运行验证	33
3.8	实现演进与核心片段	34
3.8.1	身份代次：从访问范围到生命周期键	34
3.8.2	结构化工具：从字段校验到 Execution Contract	35
3.8.3	Context：从 FIFO 截短到 active path 与 provenance	37
3.8.4	Catalog：从逐项建档到批量复用与 Typed Watch	38
3.8.5	任务委派：从 N1 MESSAGE 到 native Task Channel	41
3.8.6	workflow 服务：从线程份额到 Credit Domain 与 EEVDF	43
3.8.7	Nexus 路径演进与通用 Harness 接管	45
第四章	项目测试	48
4.1	功能与性能测试	48
4.1.1	测试组织	48
4.1.2	任务一至任务五的 Guest 综合验收	49
4.1.3	结构化工具与任务通道测量	49
4.1.4	结果文件发布与冲突处理	51
4.1.5	文件查询路径的 I/O 观测	51
4.1.6	测量数据与可重复性	52
4.2	测试路径与结果	53
4.2.1	Guest 原生工作负载	53
4.2.2	工具、执行约定与任务通道测试	54
4.2.3	长期 Guest 与 Host Harness	56
4.2.4	版本化开发与独立 Guest 运行	56
4.2.5	失效状态与恢复分支	57
第五章	总结与展望	58
5.1	阶段性结论	58
5.2	当前技术限制	58
5.3	后续工作	58
5.4	比赛收获	59
第六章	参考文献	60

1. 项目背景

大语言模型（Large Language Model, LLM）能够根据输入上下文生成文本、代码或结构化结果。AI 智能体在模型之外加入环境观察、任务规划、工具执行和结果回读，把一次回答扩展为持续运行的任务循环。当智能体进一步创建进程、读写文件、调用工具并委派子任务时，模型决策便会转化为可观察的系统副作用。身份、任务、上下文、文件和资源因而需要在执行层共享一致的生命周期与权限规则，并留下能够复盘的状态记录。

1.1. 行业背景与问题定义

1.1.1. 从内容生成到可执行副作用

ReAct 是 Agent Loop 的代表性范式：模型交替产生推理轨迹和面向环境的动作，再根据新观察修正后续判断 [1]。在 AgentOS-uCore 中，Guest Runtime 将模型选择转换为结构化工具请求，内核记录请求引发的真实副作用、上下文变化和资源消耗，使每一轮观察与行动都能关联到具体任务。

当智能体进一步创建进程、读写文件、发送消息、调用外部服务或把子任务委派给其他智能体时，模型输出便不再停留在文本窗口。一次错误判断或受污染的上下文，可能沿“模型决策-工具调用-系统副作用-新上下文”的闭环被放大；仅在提示词或应用日志中记录角色和权限，也难以保证实际执行路径采用同一套身份、生命周期和资源规则。

NIST 发布的《Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile》（NIST AI 600-1）指出，生成式 AI 风险可能出现在设计、开发、部署、运行及退役等阶段，也可能位于模型、应用乃至生态层面；该报告围绕治理、内容来源、部署前测试和事件披露等重点，给出 Govern、Map、Measure、Manage 框架下的建议行动 [2]。从操作系统视角看，这意味着执行层不仅要“把调用做完”，还应留下稳定、可核对的身份、来源、状态迁移和资源证据，使上层的测试、监测和事件复盘有可信落点。

OWASP《Top 10 for LLM Applications 2025》将提示注入、敏感信息泄露、不当输出处理、过度代理能力和无界资源消耗等列为 LLM 应用需要优先关注的风险类别 [3]。与 AgentOS-uCore 执行层设计直接相关的五类风险如图 1.1 所示，它们贯穿一次智能体行动从接收上下文、组织工具请求到提交副作用并返回结果的全过程。

OWASP Top 10 for LLM Applications 2025

Official report contents, Version 2025

LLM01:2025 Prompt Injection	3	LLM06:2025 Excessive Agency	22
LLM02:2025 Sensitive Information Disclosure	7	LLM07:2025 System Prompt Leakage	26
LLM03:2025 Supply Chain	11	LLM08:2025 Vector and Embedding Weaknesses	29
LLM04: Data and Model Poisoning	16	LLM09:2025 Misinformation	32
LLM05:2025 Improper Output Handling	19	LLM10:2025 Unbounded Consumption	35

Source: OWASP GenAI Security Project, OWASP Top 10 for LLM Applications 2025, Version 2025 (18 November 2024). Red frames are annotations added for this document.

图 1.1: OWASP Top 10 for LLM Applications 2025 及本文关注的执行层风险

图中红框从输入、数据、输出、执行权限和资源消耗五个方向说明了执行层需要处理的问题，具体体现在以下方面。

1. **输入与 LLM01 Prompt Injection（提示注入）**：受污染的指令可能改变模型选择的工具及其参数。为此，Guest Runtime 先把模型选择编码为结构化请求，内核再依据 Tool Registry、参数 schema 和调用者能力逐项检查，使自然语言决定经过明确的系统契约后才能形成副作用。
2. **数据与 LLM02 Sensitive Information Disclosure（敏感信息泄露）**：Context、文件和任务产物都可能成为后续推理的输入。项目以身份、能力位和文件作用域决定可读对象，并把数据版本与来源写入 Context 和 artifact，使访问控制与后续使用指向同一份执行记录。
3. **输出与 LLM05 Improper Output Handling（不当输出处理）**：模型回复、工具结果和系统观测都要经过解析与核验，不能直接拼接为下一次系统调用。带版本的定长结构、返回值约定和请求关联号用于识别结果属于哪次调用，Guest 只把已经结算的 TOOL 结果送入下一轮 Context。
4. **执行权限与 LLM06 Excessive Agency（过度代理能力）**：智能体能够创建任务并调用工具后，授权必须随身份和生命周期进入实际执行路径。AgentOS-uCore 通过 Agent 身份、代次、能力位、消息路由、Execution Contract 和 Task Channel 状态机共同约束委派、领取、完成与取消。
5. **资源消耗与 LLM10 Unbounded Consumption（无界资源消耗）**：循环调用、并发子任务和长时间等待会持续占用 CPU、队列与内存。 workflow 资源账户汇总用量，固定容量队列、超时和取消控制未完成请求，EEVDF 调度则按 workflow 实体分配 CPU 服务。

进一步地，OWASP《Agentic AI – Threats and Mitigations》v1.1 将智能体的规划与推理、记忆与状态、工具使用以及单智能体和多智能体交互放入同一参考架构，并分别讨论身份、工具、记忆和协作链路中的威胁与缓解思路 [4]。这些风险延伸到实际执行层：即使模型服务本身不变，权限继承、任务取消、结果归属、上下文来源和资源结算仍需要稳定的系统机制。

1.1.2. 智能体执行的基础对象与风险链条

从系统实现看，Agent Loop（智能体循环）将若干传统对象组织成更长的因果链：模型给出决策，用户态运行时把决策翻译为结构化请求，内核和设备执行副作用，结果再成为下一轮上下文。为了约束和复盘这一链条，至少需要区分表 1.1 中的四类对象。

基础对象	需要回答的问题	只留在应用文本中的风险
身份与权限	当前行动属于哪个智能体、角色和工作流代次，能够访问哪些对象、调用哪些工具	标识符复用后误认旧主体；子进程或代理链权限漂移；不同入口采用不同授权判断
工具请求与副作用	请求使用何种参数结构、由谁执行、结果对应哪次请求，何时算作提交	松散文本在各层重复解析；错误结果与新请求错配；重试导致副作用重复发生
上下文与来源	本轮决策读取了什么、数据来自哪里、文件及工作区处于哪个版本	过期或受污染状态被持续复用；结果与输入版本脱节；故障后无法重建因果链
任务与资源	子任务由谁委派、处于何种状态、取消或终态如何传播，工作流消耗多少资源	任务重复认领或完成；取消停留在消息层；通过增加线程放大调度份额

表 1.1: 智能体执行链上的基础对象

其中，能力（**capability**）表示内核允许主体执行的一组动作，代次（**generation**）用于区分同一数值标识在不同时期代表的主体，来源追溯（**provenance**）记录数据和结果的因果来源，终态（**terminal state**）则约束完成、失败和取消按照明确状态机发生。它们共同回答“谁在什么版本上，以何种权限完成了哪次可观察动作”；内容真伪和自然语言推理仍由上层判断。

1.1.3. 现有方案的特性与适用范围

Model Context Protocol（MCP）以 Host、Client 和 Server 三类组件及 JSON-RPC 消息统一模型应用访问工具、资源与提示模板的方式，由服务端声明能力，客户端按协商结果调用。Agent2Agent Protocol（A2A）则面向智能体之间的协作，通过 Agent Card 描述能力，并以 Message、Task 和 Artifact 表达委派过程与产物 [5, 6]。两者为本项目提供了重要的接口参照：Tool Registry 与 schema 采用显式能力声明，Task Channel 将任务、状态和结果关联落实为内核对象，使结构化请求进入本机执行后仍有统一记录。

除互操作协议外，业界还形成了应用编排、进程隔离、访问控制和工作流持久化等多类基础设施。表 1.2 从系统执行层比较其主要特性和适用范围。

方案类别	主要特性	系统执行层仍需补足的能力
应用层智能体编排器	提供状态图、记忆、重试、人工确认和多智能体协作，便于快速实现业务流程	策略通常依赖单个运行时；当动作跨越进程、系统调用或不同工具服务时，身份和终态容易被多份状态分别维护
MCP、A2A 等互操作协议	提供公开的能力描述、结构化消息和任务产物组织方式，降低框架与服务之间的接入成本	协议定义通信契约，但不替本地操作系统裁定进程权限、VFS 访问、任务代次、取消后的资源回收和 CPU 公平性
容器、沙箱、ACL 与 cgroup	提供成熟的进程隔离、访问控制和粗粒度资源上限，适合隔离不同服务或租户	通常以进程、容器或用户为主体，难直接表达一次工具调用的 schema、智能体角色代次、上下文来源及跨进程工作流的统一结算
数据库、向量库与工作流引擎	提供持久化、检索、检查点和失败重试，适合保存业务状态和长期记忆	需要把进程生命期、文件版本和任务终态复制到外部存储；崩溃清理、版本绑定和跨存储因果核对仍需额外协议

表 1.2: 现有方案的能力与系统执行需求

应用层负责模型编排、业务策略、持久化知识库和网络协议，操作系统执行层则统一管理进程权限、工具参数契约、任务代次、上下文因果关系和资源结算。由上层决定“做什么”，由公共执行层稳定回答“谁被允许做、动作属于哪次任务、造成了什么副作用、消耗了多少资源”。

1.1.4. AgentOS-uCore 的系统方案

LearningOS 是面向系统软件教育的开源社区，uCore-Tutorial-Code-2025S 是其采用 C 语言实现的教学内核项目，按实验章节覆盖启动、进程、虚拟内存、文件系统、并发和系统调用等内核主干 [7]。它提供能够启动并运行用户程序的完整内核，同时保留便于理解和修改的代码结构。AgentOS-uCore 直接扩展其中真实的 PCB、页表、VFS、IPC 和调度对象，使新增机制与普通 uCore 程序共同运行。

RISC-V 是一种开放、模块化的指令集架构。其非特权规范以基础整数指令集为核心，可按需组合原子操作、压缩指令等标准扩展，特权规范则定义陷入、页表和特权级；RV64 表示整数寄存器宽度 XLEN 为 64 位的一组基础架构 [8, 9]。本项目沿用 uCore 的 RISC-V 启动、页表与异常处理路径，内核与 Guest Runtime 均面向 RV64 编译，因此进程创建、页表切换、系统调用和陷入处理都经过目标指令路径，功能与性能证据也来自同一架构。

在上述基础上，AgentOS-uCore 保留普通进程、虚拟内存、VFS、异常处理和系统调用主干，在现有内核对象与调用路径中补充智能体身份、上下文、结构化工具调用、任务通道、来源记录、资源账户和工作流调度实体。新接口使用定长且带版本和结构大小的 UAPI；普通 uCore 进程仍走原有路径，智能体进程则通过受控的创建和 exec 路径取得内核身份。

图 1.1 中的五项风险最终被转化为三类执行要求：每个动作都能定位到身份与 workflow 代次，每次副作用都经过结构化契约和任务状态机检查，每轮执行都留下 Context、来源与资源记录。上层据此判断提示语义、数据敏感性和业务目标，内核则在真实进程、文件、消息和调度路径中执行统一规则。

项目把多个应用都会重复实现、又必须在真实副作用发生处统一裁定的机制下沉到 uCore，在执行层提供最小权限、明确终态、可追溯证据和有界资源。模型服务、传输加密和密钥管理继续由 Host 承担，应用层负责模型输出与业务语义。

1.2. 核心挑战与设计目标

AgentOS-uCore 的核心挑战在于让身份、任务、上下文、文件和调度在同一条执行链上对“主体与代次”给出一致答案。项目同时保留普通 uCore 工作负载的原有语义，将文件选择与跨轮历史留在 Guest，并用 Guest 行为、Host 记录和固定工作负载共同验证。表 1.3 将这些要求整理为“挑战-设计目标-验证路径”。

比赛指定 QEMU RISC-V64 virt 作为运行平台。本项目为每轮测试启动一个全新的 AgentOS-uCore Guest，通过串口收集启动、故障和性能记录，并将固定工作负载的 Host 与 Guest 结果相互核对。后文的 Guest 测试由此覆盖实际内核路径上的冷启动、系统调用、故障处理和资源结算。

EEVDF（Earliest Eligible Virtual Deadline First，最早可服务虚拟截止时间优先）是一种比例共享调度算法。它先根据滞后量判断实体是否具备服务资格，再从合格实体中选择虚拟截止时间最早者；实际服务时间会回写虚拟运行时间，使权重和等待状态持续影响后续选择 [10, 11]。AgentOS-uCore 将这种调度思想作用于整个工作流，让同一工作流中的进程共享 CPU 服务记录，并在选中工作流后再由原有 uCore 策略选择具体线程。

核心挑战	设计目标	验证路径
身份与权限跨对象保持一致	以 <code>id+generation</code> 作为内核主体，角色、能力位和控制标识随受控的 <code>create</code> 、 <code>fork</code> 、 <code>exec</code> 、 <code>exit</code> 路径迁移；工具、VFS、IPC 与任务路由使用同一身份判定	同时运行普通进程与智能体进程；覆盖创建、继承、映像替换、退出和标识复用；检查越权、过期代次和错误 <code>route</code> 均被拒绝
任务委派具有唯一、可取消的终态	由原生 <code>Task Channel</code> 的 <code>SQ/CQ</code> 和内核状态机维护 <code>submit</code> 、 <code>claim</code> 、 <code>complete</code> 、 <code>cancel</code> 及背压，避免把终态仅作为应用消息约定	构造并发认领、重复完成、取消竞态、队列满和过期请求；核对状态码、完成项及资源回收结果
上下文、来源与工作区版本可以复盘	以七页 <code>Context</code> 区承载容量受控的活动路径，以 <code>dev+inum+incarnation</code> 绑定启动期文件元数据、索引和订阅； <code>Host</code> 只在会话开始时固定的 <code>root</code> 范围内提供 <code>manifest</code> 与版本字节，不替 <code>Guest</code> 选择文件或改写 <code>Provider</code> 历史	对比遍历、属性查询和索引结果；触发增量缺口后的重同步；检查 <code>artifact</code> 、 <code>Context</code> 、 <code>manifest</code> 与模型请求记录的版本和关联号
资源结算抵抗线程扩张并支持事件驱动	以工作流为资源与调度实体，汇总 <code>CPU</code> 等资源，用原子等待替代轮询，并按 <code>EEVDF</code> 的滞后量和虚拟截止时间分配服务	比较单线程与多线程工作流的服务份额；验证无事件时休眠、事件到达后唤醒；报告等待时延、吞吐与资源账户
扩展保持兼容且证据可重复	保留普通 <code>uCore</code> 路径；所有新增 <code>UAPI</code> 使用定长、版本化结构；功能与性能样本绑定相同输入、工作区清单和结果指纹	运行基础回归、静态检查与 <code>QEMU Guest</code> 测试；使用固定回放核对 <code>Host/Guest</code> 证据；性能比较拒绝指纹不一致的样本

表 1.3: AgentOS-uCore 的核心挑战、设计目标与验证路径

这些目标最终落实到三类可检查对象：内核保存的状态、触发状态变化的调用路径，以及覆盖正常与异常分支的测试结果。后续章节依次展开 `Host`、`Guest` 与内核的职责分工，并说明身份、`Context`、结构化工具、任务通道、VFS 查询、IPC、资源和调度的实现与验证。

2. 系统设计

AgentOS-uCore 沿用 uCore 的进程、页表、VFS、异常处理、系统调用和调度主干，并直接扩展现有内核对象。一个智能体发起工具调用时，身份检查、上下文记录、文件访问、消息投递、资源记账和调度选择都读取进程中保存的同一组 workflows 编号和代次。图示、数据结构和状态机明确规定这些模块何时读取字段、何时更新状态，避免同一请求在不同位置被认作不同 workflow。

2.1. AgentOS-uCore 总体架构

2.1.1. 以 uCore 为底座的产品分层

AgentOS-uCore 的总体架构如图 2.1 所示，其核心由 Host Relay、Guest 应用与 workflow、Agent UAPI 与 Guest Runtime、AgentOS 内核功能层和 uCore 内核基座五部分构成。一次任务从用户目标或模型回复开始，经 Guest 应用组织为工具调用或子任务，再通过统一 UAPI 向下进入内核；执行结果、Context 记录、来源信息和资源用量沿相反方向返回 Guest，形成下一轮模型决策所需的观察。图中主体自上而下展示 Guest 到内核的执行路径，左侧 Host Relay 则连接模型 Provider、版本化工作区与 QEMU 串口。

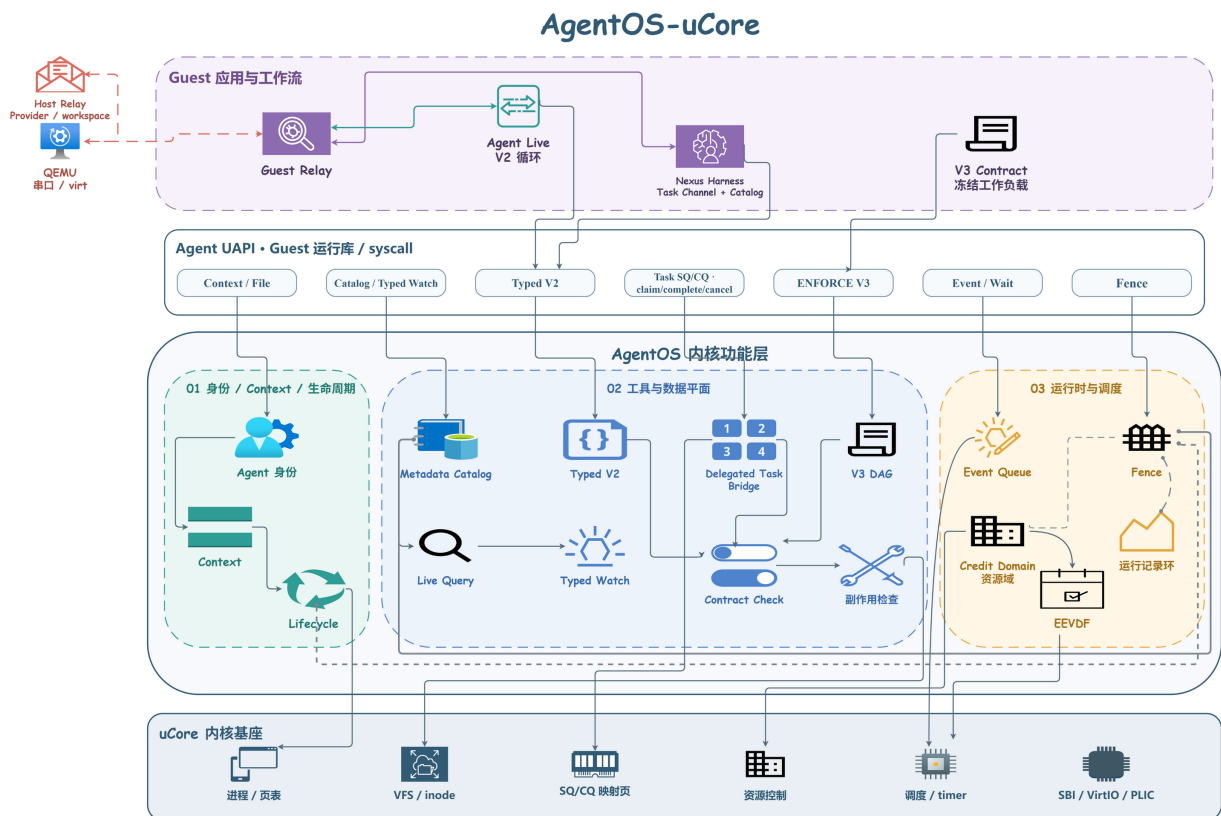


图 2.1: AgentOS-uCore 总体架构

五部分依照信息来源、业务决策、系统契约、执行裁定和基础资源管理逐层协作。各部分的设计思路如下。

1. **Host Relay:** 该部分运行在 QEMU Guest 之外，负责 TLS 连接、模型 Provider 格式适配、版本化工作区字节传输和串口帧转发。网络密钥、远端服务差异与宿主文件访问由 Host 处理，使内核专注于本机对象与执行状态；Provider 回复和工作区数据进入 Guest 后，再与具体任务、文件 revision 和 Context 记录建立关联。
2. **Guest 应用与 workflow:** Agent Live V2、通用 agentharness_ucore 和 V3 Contract 工作负载位于这一层。它们理解用户目标与模型回复，决定何时直接回答、调用工具或委派子任务，并维护会话和业务步骤。控制台与在线模型都通过同一套内核机制执行，因此业务策略可以变化，身份、任务终态和资源结算的含义保持一致。
3. **Agent UAPI 与 Guest Runtime:** 这一层把上层选择转换为带版本、结构大小和类型信息的请求，统一封装运行时配置、Context、Artifact metadata、查询预测、单条与批量 Metadata、Catalog、Typed Watch、Task SQ/CQ、Execution Contract、事件等待与 Workflow Fence 等系统调用。Guest Runtime 可以根据预计查询次数选择目录遍历或批量建档，并在同一 lifecycle 内复用已经核验的 Catalog；调用向下提交，逐项状态、完成项和查询结果再由此还原为 Guest 可使用的对象。
4. **AgentOS 内核功能层:** 内核功能按照实际裁定位置组织为三组。身份、private Context 与生命周期把一次动作绑定到具体 Agent 和工作流代次；工具与数据平面检查 schema、capability、Artifact seal metadata、文件元数据、动态 Task Channel 和副作用清单；运行时与调度处理事件等待、查询预测、资源账户、EEVDF、执行记录和结束快照。Metadata 批次共享一次生命周期操作和事务，文件变化路径则在没有任何相关 Watch 时快速结束事件发布。由此，权限检查、状态迁移和证据写入都发生在真实副作用经过的路径上，完成结果再携带 Task、Artifact 与 Context 关联返回 Guest。
5. **uCore 内核基座:** 进程与页表、VFS 与 inode、SQ/CQ 映射页、资源控制、调度与 timer，以及 SBI、VirtIO 和 PLIC 等原有机制继续负责进程运行、文件访问和设备交互。AgentOS 直接扩展这些现有对象，使智能体身份能够随进程生命周期变化，文件来源能够落到 VFS 对象， workflow 服务量也能够进入调度器的实际选择过程；普通 uCore 程序仍使用原有调用路径。

五部分共同形成完整的上下行闭环：Host 提供模型和工作区输入，Guest 决定业务动作，UAPI 固化请求形式，AgentOS 内核完成检查、状态迁移与记录，uCore 基座执行最终的进程、文件和调度操作。为了进一步说明内核在这条主线中承担的职责，下一节将其归纳为六项核心机制。

2.1.2. 六项核心机制与请求流向

上述架构按部署位置说明系统组成，表 2.1 则从一次请求需要经过的内核能力出发，列出六项相互配合的机制。

模块	关键对象	主要功能
Agent 进程与地址空间	身份记录、运行时配置、七页 Context 区、生命周期	创建受信任 Agent，绑定 capability、允许工具、资源额度和工作流代次，并把高频状态映射给 Guest
结构化交互与工具协议	33 项目录、V2/V3、128 字节 Task descriptor、SQ/CQ	发现工具、检查带类型信息的参数，并支持动态调用、冻结执行约定和有界任务提交
上下文路径与来源追溯	private path、Artifact seal metadata、共享索引、预测器	连接摘要、完整正文、Task 结果和数据来源，并从成功只读历史生成有界预取
面向 Agent 的文件查询	批量元数据、摘要、索引、订阅器	为显式登记的文件提供自适应建档、结构化查询、Catalog 复用、变更通知和缺口重新同步
Agent Loop	事件队列、路由、心跳	提供原子等待与唤醒、消息、模型请求关联、超时和取消
工作流资源与调度	资源账户、EEVDF 实体	按工作流汇总资源记账，并根据滞后量和虚拟截止时间分配 CPU 服务

表 2.1: AgentOS 的核心模块

六项机制使用同一个工作流 id+generation。一次工具调用会同时取得调用者身份、前一条上下文记录、schema、数据来源、资源账户和调度实体。工作流代次变化后，已经回收槽位对应的订阅器和句柄不能再指向新对象。

2.1.3. 一次智能体工具调用

一次完整调用从 Guest 读取工具目录开始。Guest 按当前 runtime config 中的 capability 与允许工具位图筛选可用项，再将模型结果编码为带类型信息的请求。请求进入内核后，AgentOS 依次拷入数据，检查版本和大小，检查身份与生命周期，解析工具并解码 schema，再检查执行约定、数据来源、资源和副作用，最后调用 VFS、IPC 或其他 uCore 子系统。返回前，上下文路径写入一条新记录；较长正文先进入 Artifact Store，内核封存 metadata 与 SHA-256 后再由记录引用。

当智能体等待模型、文件事件或其他 Agent 时，agent_wait 将线程置于等待状态。事件入队与等待者安装按原子顺序执行，从而避免“发现队列为空后恰好错过唤醒”。

2.1.4. 上层 runtime

控制台按固定策略依次执行身份检查、上下文记录、文件查询、IPC 和恢复操作，用于功能演示和配对测量。预计只查询一次时，控制台直接遍历目录；预计重复查询时，它批量登记 Metadata，并让同一 lifecycle 内的查询共享一个 READY Catalog。Nexus Harness 为所有 Agent 运行同一套事件驱动 Loop，CLI 只提供目标、workflow policy、工作区和可选配置；拥有 ORCHESTRATE 的 Agent 可以建立子任务，名称不参与内核授权。整个 Harness 会话共用一个长期运行 Guest，Host Agent 通过 runtime SPAWN 与原生 Task Channel 对应

到 Guest 进程和 Task。Host broker 执行版本化读写、隔离构建和独立 Guest 测试，结果封存为 Artifact 后进入 private Context 或 workflow 共享索引。

同一科研恢复任务在普通 uCore 与 AgentOS-uCore 中的执行组织如图 2.2 所示。两条路径使用相同的输入、分析、恢复和写回步骤，差异在于身份、上下文、等待关系和资源信息由应用分别维护，还是进入贯穿各阶段的内核对象。

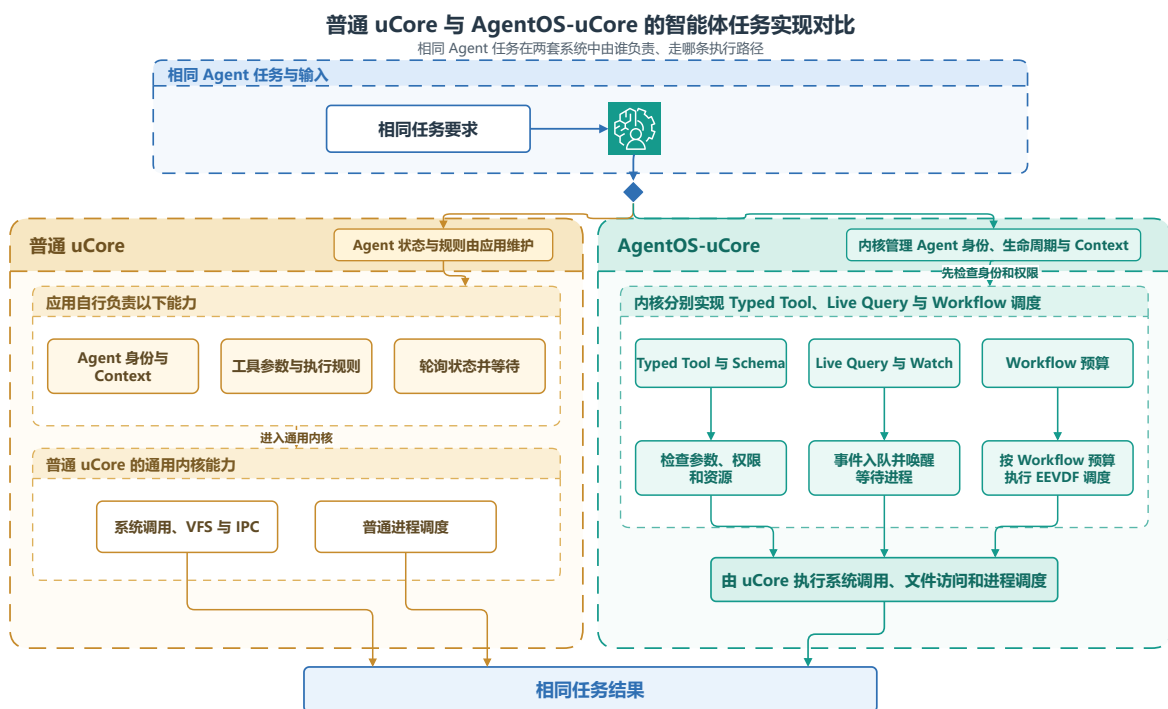


图 2.2: 同一科研恢复任务在普通 uCore 与 AgentOS-uCore 中的执行路径

普通 uCore 路径。应用使用进程、文件和 IPC 串起各阶段，同时自行保存调用者配置、调用关联、上下文摘要、等待条件和资源统计。内核提供通用机制，每次业务状态迁移及其日志关联由应用代码组织。

AgentOS-uCore 路径。应用仍负责相同的科研流程，但工作流身份、结构化工具、选择性索引、事件等待、来源记录和工作流调度成为可组合的内核机制。Guest 提交带任务关联的请求后，内核在文件访问、消息传递和调度发生处完成检查与记账，并把结果接入同一条 Context 路径。

共同结果。两条路径最终都完成数据读取、分析恢复和结果写回。AgentOS-uCore 的变化在于把跨进程、跨工具反复需要的执行状态交由公共机制维护，使同一业务结果同时带有可核对的身份、数据来源、任务终态和资源记录。

2.2. 跨模块状态约束

系统将 workflow 标识写为可复用槽位的 `id+generation`。模块使用该标识前，先确认对象仍在运行、访问范围相同且代次匹配；资源、元数据和执行记录也按该标识归属。这样，旧槽位关闭、回收并再次使用后，之前的请求不会被新 workflow 接收。

算法 1：workflow 标识的跨模块校验

输入：调用进程、目标对象、请求携带的 workflow 编号和代次、访问范围。

输出：可继续执行的对象引用，或明确的 `STALE`、`DENIED` 结果。

1. 从调用进程读取内核保存的 workflow 编号、代次和能力位。
2. 检查 workflow 编号和代次的结构，确认生命周期仍处于活跃状态并且代次匹配。
3. 比对调用者、目标对象和请求的访问范围；如不一致，则拒绝提交副作用或传递事件。
4. 在工具、元数据、任务通道、执行记录或调度路径中，分别执行相应的参数和资源检查。
5. 只有在全部检查通过后，才发布状态、将事件入队或提交资源。

3. 系统实现

本章结合当前仓库的内核与用户态源码说明各项机制的实现路径。内核实现集中在 `os` 目录下的 `Agent`、`workflow` 与资源控制模块，用户态场景位于 `user/src`；`Nexus` 还使用 `Guest Runtime` 和 `Host Relay` 完成跨环境协作。

实现部分	主要源文件	关键责任
启动与身份	<code>main.c</code> <code>agent_identity.c</code> <code>workflow_*.c</code>	初始化智能体子系统，分配 <code>capability</code> 、工具集合、控制标识和工作流生命周期。
上下文与执行	<code>agent_context*.c</code> <code>agent_core.c</code> <code>agent_execution*.c</code> <code>agent_task*.c</code>	写入 <code>private Context</code> ，封存 <code>Artifact metadata</code> ，维护查询预测，并按执行约定检查副作用。
任务与来源	<code>agent_provenance.c</code> <code>agent_evidence*.c</code> <code>agent_metadata*.c</code>	管理动态任务描述符、 <code>SQ/CQ</code> 、来源标签和工作流执行记录的阶段快照。
文件与事件	<code>agent_file*.c</code> <code>agent_live_query*.c</code> <code>agent_ipc.c</code>	维护元数据目录、查询与订阅、结果文件发布、消息和等待。
资源与调度	<code>resource_controller.c</code> <code>workflow_credit*.c</code> <code>workflow_scheduler.c</code>	处理资源使用租约、额度、工作流级选择和资源记账。
集成场景	<code>labdemo_ucore.c</code> <code>agentharness_ucore.c</code> <code>agentmulti_ucore.c</code>	驱动恢复流程、长期 <code>Guest</code> 原生 <code>Task Channel</code> 、动态 <code>Agent</code> 配置、 <code>Artifact</code> 和预取验收。

表 3.1: AgentOS-uCore 的实现源代码分工

3.1. Agent 进程创建与地址空间设计

3.1.1. 从普通进程到工作流中的 Agent

`AgentOS` 区分普通进程、协调 `Agent` 和专职 `Agent`。`VFS`、`IPC`、工具执行器和调度器读取同一份身份记录，因此同一个进程在各处拥有相同的角色和能力位。内核将 `Agent` 身份写入 `PCB`，再根据受信映像清单和父角色授权完成创建。用户态可以读取身份信息，权限判断始终以内核副本为准。普通进程仍走 `uCore` 原有的创建与装入路径，两类进程可以在同一系统中并存。

创建受控工作流时，内核先分配生命周期，再写入身份、角色、能力位和控制标识，随后建立 `Agent Context` 页、`VFS` 访问范围和资源账户。任一步骤失败，内核都会回收已经分配的对象。创建完成后，各子系统都通过同一组工作流编号和代次找到该 `Agent` 所属的工作流。

用户态的 `agent_create()` 和 `agent_create_role()` 进入内核的 `Agent` 创建路径；`agent_info()` 返回身份、角色、配额、`Loop` 状态和 `Context` 位置，另一查询接口返回完整生命周期键。创建接口负责建立对象，查询接口只复制内核已经保存的元信息，两者都不会让用户态自行填写权威身份。

`agent_worker_create()` 不会新建工作流根生命周期。它只在已有生命周期中创建一个携带受控待执行映像的子进程；调用者只要在同一工作流内具有 `ORCHESTRATE` 即可调用，不要求它必须是控制者。子进程仍须沿通常的 `exec()` 路径装入映像，所以该调用返回时，子进程尚未在新映像中运行。`agent_scope_delegate_fd()` 只接受 `FD_INHERIT_DELEGATE`，当前实现用于管道。该接口先在调用线程上登记一次性凭据，下一次创建受控子进程时再由内核制作创建快照并使用该凭据。子进程通过 `filedup()` 获得文件描述符副本，父进程继续保留原描述符，随后内核清除这张凭据。

3.1.2. 身份字段与角色策略

字段	含义	使用模块
<code>agent_id</code>	当前启动周期内的智能体编号	工具、上下文、观测
<code>control_id</code> 角色与能力位	内核分配的控制身份角色及其具体权限集	控制者绑定、IPC 路由、句柄校验 工具、文件、消息与 <code>artifact</code> （工作流结果文件）操作
工作流 <code>id+generation</code>	工作流标识和可复用槽位的代次	生命周期、订阅、任务通道、资源、调度
VFS 访问范围	工作流的文件访问范围	元数据、查询、 <code>artifact</code>
执行与存储账户	工作流资源归属	进程、线程、文件、块、页与缓冲区

表 3.2: 智能体身份的核心字段

内核的角色策略将每个角色映射为能力位掩码和调度权重。哨兵角色主要读取进程、元数据和调度状态；调查角色只能读取受限内容；报告角色可以创建报告结果文件；协调角色可以分派任务并创建角色。子智能体继承同一工作流的访问范围，其能力位同时受父角色授权和映像清单限制。

3.1.3. Agent Context 区的七页布局

每个 Agent 在用户地址空间的固定 ABI 地址拥有七页 Agent Context 区。前六页由内核维护，并以只读方式映射到 Guest，依次提供身份头、最近返回、上下文记录、路径索引和稳定摘要；第七页可由用户态读写，用于保存查询结果和其他高频策略数据。这样，身份、能力位、访问范围等权威状态仍受内核保护，Guest 读取 Context 和管理查询缓存时又不必为每个字段反复进入内核。

上下文最多保存 128 条记录。每条记录包含序号、请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识和记录摘要。内核按发布序号更新镜像；Guest 既可以调用查询与快照接口，也可以直接读取同一结构。PCB 保存身份、配额、Loop 状态和路径位置等低频元信息，具体路径则通过 Context 区向用户态发布，这一划分兼顾了内核裁定与高频读取。

3.1.4. 生命周期代次与并发控制

工作流槽位可在回收后复用，内核每次重新分配时都会推进代次。订阅器、资源账户、VFS 附属记录、任务句柄和 `artifact` 都同时保存标识和代次。

生命周期使用普通操作、退出操作和 **Workflow Fence**（工作流栅栏）三类并发控制。普通系统调用增加普通操作计数；`exit` 或拆除过程增加退出操作计数；**Workflow Fence** 在两类操作全部完成后生成阶段快照。当控制者已经关闭、成员数、活跃普通操作数和退出操作数都归零，并且栅栏已经释放时，回收程序会清除该生命周期的上下文、订阅器和执行记录环。普通 VFS 访问范围会被完整回收；对于 `preserve_on_retire` 持久输出访问范围，只要存储账户进入 `DRAINING` 或 `FREE`，内核即可回收生命周期代次，同时保留登记项、文件和存储额度。

3.1.5. 随工作流创建的资源主体

工作流创建时，内核同时分配执行账户和存储账户。工作流中的进程、线程、文件对象、文件系统块、`inode`、智能体状态页和物理页都记入同一账户。这样，资源使用租约和工作流 `EEVDF` 都按工作流计算，子进程和多线程也始终记在原工作流名下。

3.1.6. 验证结果

`agenttrust_ucore` 覆盖受信 `exec`、角色授权、能力位收缩以及 `fork`、`exec`、`exit` 与普通进程共存；`agentscope_ucore` 覆盖访问范围与代次复用；`agentfinal_ucore` 覆盖上下文固定映射、快照、分支、回滚和容量淘汰。所有场景都在 `RV64 uCore Guest` 中通过真实系统调用运行。

3.2. Agent-OS 内核结构化交互接口

3.2.1. 工具调用协议与工具目录

Agent 既要发现可用能力，也要把工具名称、参数和返回状态交给内核可靠解析。AgentOS 为此定义带版本和大小字段的工具调用协议，`Guest` 只传递结构化请求。内核维护 33 项工具目录，每个描述项写明工具编号、名称、`schema`（参数模式）、标志和副作用类别。目录中有 20 项可调用工具、3 项 `syscall-only` 工具、3 项废弃兼容编号和 7 项 `Host broker` 工具；第 26 项是原生 `delegate_task`，第 27–33 项覆盖补丁、写入、搜索、读取、系统观察、隔离构建和独立 `Guest` 运行。启动时检查编号与名称是否唯一，并检查 `schema` 是否完整。

接口整理后，`rerun_stage`、`write_report` 与旧 `ctx_stat` 调用统一返回 `DEPRECATED`。动作与 `Artifact` 更新分别使用 `action_commit` 和 `artifact_update`；`Context` 状态、`Heartbeat` 配置和 `Metadata` 初始化分别使用 `context_status`、`heartbeat_configure` 与 `metadata_init`。ABI vNext 将公开 `UAPI` 版本提升为 2，删除恒零 `mailbox` 字段、V1 固定请求结构、系统调用 503/504 和旧 C 包装，退役编号保持为空。`apply_patch` 和 `write_file` 采用独立编号，要求 `AGENT_CAP_WORKSPACE_WRITE`，并将 `lifecycle`、`Host manifest object`、预期 `revision`、路径和内容 `Artifact` 绑定到 `workspace mutation` 描述符。构建与运行也使用 `brokered Registry` 条目。普通 V2/V3 执行返回 `BROKER_REQUIRED`；`Nexus Host broker` 核对 `workspace root`、固定路径模式、准确 `revision` 与内容长度，随后执行原子提交、受限构建或独立 `Guest` 测试。

通用运行时配置包含 `capability`、允许工具位图、提示词 `Artifact`、资源预算、`Artifact` 数量/字节/读取额度和摘要高水位。拥有编排权限的父 `Agent` 只能向子 `Agent` 授予自身与用户 `workflow policy` 共有集合的子集，`Agent` 名称不参与授权判断。

请求使用工具名称或编号加一组 `typed` 键值参数，响应返回状态码、`Context` 序号、结果类别和定长结果字段。版本不符、参数类型错误、工具不存在、能力不足和生命周期过期分别返回可区分的状态；用户态不需要从一段错误文本中猜测失败原因。目录项和请求结构都带 `version` 与 `size`，后续扩展可以在明确拒绝旧布局的前提下进行。

`sys_tool_list` 向 `Guest` 返回定长描述项。`Guest` 再按 `runtime config` 中的 `capability` 与允许工具位图列出模型可见项，并补充用途、参数和结果说明。等待、上下文和模型请求等 `runtime`（运行时）接口只供 `Guest` 使用，模型只能请求当前 `Agent` 配置允许的产品动作。

3.2.2. 自适应的 V2 调用

V2 请求明确携带版本号、请求大小、工具名称、工具编号、参数数量和带类型信息的参数数组。当前标量类型为 `uint64` 与字符串，适合 `uCore` 的紧凑参数解析。内核先把完整参数拷入内核缓冲区，依次检查参数名、类型、数量和长度，再进入共享执行器。

V2 适合根据上一轮结果改变下一步操作的智能体循环。例如，协调智能体可以先查询失败阶段，再根据候选数决定读取摘要还是创建研究子任务。每次调用都会独立检查身份、能力位、访问范围、`schema`、数据来源和资源状态。

3.2.3. 四种执行路径

路径	输入形式	主要用途	运行语义
紧凑批处理	最多 64 个	固定顺序和上下文压力	在一次系统调用内顺序
V2	<code>agent_op</code>	场景	执行，逐项返回状态
	带类型信息的键	依据上一轮结果选择工	每次都检查 <code>schema</code> 、能
	值参数	具	力位和访问范围
ENFORCE V3	V2 前缀与	预先定义的高保障 DAG	冻结节点、前驱、尝试
	<code>Execution Contract</code>		次数、截止时间和副作用
	绑定信息		清单
任务通道的	16 槽 SQ 与 16 槽	有界提交、完成和背压	仅主线程建立通道，绑
SQ 与 CQ	CQ		定提交线程并校验代次；
			任务进入终态结算时至
			多发布一条完成 CQE，
			取消不会额外产生 CQE

表 3.3: 四种工具执行路径

紧凑批处理用于同步执行短操作序列。V3 则在工作流生命周期内冻结一份带版本的执行约定，其中最多包含 24 个按拓扑顺序排列的节点。每个节点关联工具、`schema` 摘要、所需能力位、输入和输出 `artifact` 类型、截止时间、尝试次数、来源与副作用清单，以及执行和存储额度。

V3 请求同时绑定执行约定代次、节点、尝试次数、来源节点、前驱上下文序号和 32 字节指纹。无论成功还是失败，完成结果都会写入固定完成槽；合法重试直接读取原结果，避免再次产生副作用。

3.2.4. 工具执行期间的资源使用租约

高风险工具可以在执行前从 workflow 账户预留执行和存储额度。资源使用租约依次经过 ADMITTED-→ACTIVE-→DEACTIVATED-→SETTLED。分配器在发布对象前取得 `nonce` 声明，发布成功的对象继续占用同一份已用额度，析构时再结算。这样，多个对象在同一轮工具调用中同时达到资源峰值时，账户仍能准确记录用量。

3.2.5. 带类型信息的任务通道

任务通道只能由智能体主线程建立，并映射 16 槽 SQ 和 16 槽 CQ，两者各占一个 Guest 映射页；复制的请求和完成记录、权威资源状态则分别保存在两页内核私有页中。SQE 与 CQE 均为 128 字节。进入通道和资源绑定会记录提交线程标识及身份代次。内核先拷入完整 SQE，再检查进入接口的 ABI、递增请求编号、执行约定、节点、尝试次数、schema、截止时间、取消链接和带类型信息的句柄。提交者、所有者或生命周期不匹配导致 STALE 时，输出只保留 ABI 版本、结构大小和 `status`，其余字段全部置零；控制标识代次不匹配时，进入接口返回当前通道状态。

内核私有的头尾指针才是权威状态。CQ_FULL 只表示 CQ 暂无可写槽位，是背压，不表示已接受任务已经完成；对应任务仍待发布，直到 CQ 出现可用槽位。SQE 的环或槽位代次不一致，或者发生其他协议错误，都会使通道进入需要重新同步的状态；提交者必须以 AGENT_TASK_CHANNEL_ENTER_F_RESYNC 和空 `sq_tail` 显式复位，随后才能继续提交。已接受任务在成功、拒绝、失败、取消或超时形成可交付终态后，至多发布一条完成 CQE。CANCEL 命令使用独立请求编号，只引用目标任务，不会另行生成 CQE；同步策略拒绝仍会消费取消编号，并由进入接口返回 DENIED，目标任务继续保持原运行状态。目标任务已经确认或不存在时，同样会消费取消编号，进入接口返回当前代次和水位线。Task Bridge 既接受空输入，用于验证通道、截止时间、取消和完成语义；也允许 ECHO Provider（服务提供者）读取 UTF-8 资源 snapshot，检查带类型信息的输入能否进入同一个工具执行器。

任务输入也可以由 IMPORT 控制请求从当前进程的只读普通文件描述符导入。请求进入时，内核先固定文件引用和当次 workflow/contract cut；实际读取通过带租约的 `readi` 从 offset 0 多探测一个字节。只有文件恰好在给出的 1-63 字节处结束、内容不含 NUL 且 UTF-8 合法时，才建立资源。导入提交前，内核在同一条事务 lane 中再次核对 Context 序号、哈希和来源标签，避免把文件内容与另一时刻的 Context 拼成一条来源记录。

导入过程不改变文件描述符的共享 offset，也不把 `structfile*` 留在通道中；内核只在私有资源槽保存不可变的 inline snapshot、SHA-256、最新 Context 序号和文件数据来源标签。IMPORT 只登记输入，不另行追加 Context 记录。

资源句柄由 slot、type 和 generation 共同标识。BORROWED 输入执行后仍保持 LIVE，可由后续任务继续引用；OWNED 输入在完成后自动消费，显式 RELEASE 也会使旧句柄变为 STALE。ECHO Provider 从内核 snapshot 复制 payload，不再依赖导入文件的后续内容或 offset。若 IMPORT 已成功而最终结果无法复制回用户态，内核会撤销尚未暴露的资源，避免只有内核知道的槽位泄漏。

3.2.6. 原生 delegated Task Channel

跨 Agent child Task 使用准确的 128 字节 Task descriptor。描述符绑定目标 identity、parent task、目标描述 Artifact、输入 Artifact、所需 capability、允许工具、workspace revision、资源

预算、deadline 和预期结果类型，大段输入与结果保存在 Context Artifact Store。任务委派 SQE 进入 Execution Contract 和内核 pending 队列后，内核核对父子授权包含关系、目标领取能力和独立 Task route；self delegation 与任务图环路被拒绝，多个独立子任务可以并行。

目标通过系统调用 agent_task_delegate_claim（编号 567）取得不可变描述符，先封存结果 Artifact，再通过 agent_task_delegate_complete（编号 568）提交终态。claim 建立的 delegated effect lease 精确绑定执行者 PID、Agent、control_id、线程 identity generation 以及 channel/request/slot 代次；每次直接作用必须是 Execution Contract 副作用清单的子集。正常完成、协作式终态确认或目标进程 quiescence 后，内核才在活动引用归零处撤销该租约。父 Agent 从 terminal CQE 取得结果 handle，并复核 producer、Task、Context sequence、lifecycle 和 SHA-256 后接纳结果。

同一活动生命周期内的 controller 也可复用 complete 接口，通过取消标志、CANCELLED、零 ACK 字段和准确的 owner/channel/request/slot/task/correlation tuple 请求取消。内核除核验该完整绑定外，还要求调用者具备编排与等待取消能力，并具有指向 owner 的活动 Task route。返回 OK 表示取消请求已被接收，任务终态仍以 terminal CQE 为准。

若该取消、截止时间、owner 退出或生命周期关闭在 claim 后先取得终态优先级，complete 返回带 generation 的终态 offer。处于 CLAIMED 的 provider 须先清理预绑定输出，再以 ACK_TERMINAL 和准确 offer 确认；内核随后只向 owner 发布一条 terminal CQE。当前委派协议采用协作式终止，无法强制终止永久无响应的 provider。一般 MESSAGE 路由仍用于进程通信，child Task 的正文、状态和结果由 Task Channel 与 artifact 承载。

3.2.7. 验证结果

agenttoolabi_ucore 覆盖 33 项目录发现、名称与编号一致性、统一接口、废弃状态、brokered 状态、未知参数、类型错误和返回缓冲区；agentcontract_ucore 覆盖 DAG、截止时间、重试、取消、来源 Context 与来源检查；agenttask_ucore 检查 128 字节 TASK snapshot、父子授权、多子任务图、TASK route、claim/complete、controller 取消、终态 offer 和唯一 CQE；agentmulti_ucore 检查通用运行时配置、Artifact seal/bind/share 与查询预测。最终 copyout 失败后的回滚分支由结构检查、mutation 测试和资源表模型核对。任务通道的测量数据另保存了 96 条每条 16 个操作的序列，第 4 章集中分析其分布。

3.3. 上下文路径管理、数据来源与 Workflow Fence

3.3.1. 从多轮调用到 Context Path

Agent 的下一步决策往往取决于前几轮工具结果。上下文路径管理把这些请求和结果按前驱关系组织为 Context Path：每次工具调用即将执行可能产生副作用的操作时，AgentOS 建立一条上下文记录，并由附属记录保存归一化的 agent_op、结果和 provenance（来源追溯）信息。附属记录只保存后续决策需要的请求、结果、因果关系和数据来源，完整的 V2 参数、V3 绑定与任务通道 SQE 仍由各自协议对象承载。

记录序号只在同一次清空周期内递增。记录包含请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识、短载荷或结果以及记录哈希；执行清空后，序号从 1 重新开始。父记录序号指向当前活动头记录，后继索引用于查询分支。

上下文追加依次经过准备、填充、发布和提交；detail 会在记录标为奇数前写入。镜像头发布记录数、头记录、最早记录、最新记录、可见头记录、丢弃数和回滚数。Guest 读取记录后重新核验序号和哈希，并再次确认摘要未变化，便可得到一致快照。

用户态通过 `context_push()` 追加记录，也可以用 `context_query()` 复制活动路径；高频读取时则直接检查 Context 镜像。`context_rollback()` 把可见头移到仍被保留的记录，`context_clear()` 清空路径。两种读取方式使用相同的序号和摘要，可以互相核对。

3.3.2. 分支、回滚与有界淘汰

上下文路径最多保留 128 条记录。容量用尽后，内核按序号淘汰最早记录，并更新最早记录、最新记录和丢弃计数。活动路径只引用仍然保留的前驱记录。

回滚将 `visible_head` 移到仍然存在的上下文序号，并为后续调用分配新的分支代次。已经产生的文件、IPC 和模型服务副作用保持不变。需要幂等的副作用由 V3 尝试缓存和具体工具协议处理。因此，上下文分支只改变后续决策看到的路径；手工备注、回滚和清空都不会预留或提交执行记录。

3.3.3. Context Artifact Store 与多 Agent 共享

短 Context record 保存状态、因果关系与哈希，完整 USER、TOOL、FINAL、文件、搜索结果、补丁、编译诊断、运行日志、测试结果和子任务报告进入用户态 Context Artifact Store。单项正文不超过 64 KiB，默认 workflow 总额度为 128 项和 2 MiB，模型单次投影不超过 12 KiB。内核通过 syscall 571 核验类型、长度、UTF-8、producer 与 SHA-256，并封存 handle、Task id、来源 Context sequence 和 lifecycle；封存完成后才允许 Context 节点或 Task 终态引用。

每个 Agent 保持独立 private Context。父 Agent 只在收到成功 terminal CQE 后接纳子任务结果，并重新核对 producer、Task、来源 sequence、lifecycle 和内容哈希。workflow 共享索引保存任务图、公共 Artifact、revision、build、测试、冲突和未完成 Task，只引用已经结算且明确共享的对象。rollback 只移动当前 Agent 的 active path；已经被父任务接纳或其他 Agent 引用的 Artifact 继续保留，独占且失去引用的对象按 lifecycle 延迟回收。

private Context 达到配置高水位后，Nexus 生成结构化摘要，记录目标、完成工作、工具、文件 revision、最近错误、待办和计划；workflow 摘要保存任务分解、Agent 配置、依赖、公共 Artifact、build 与测试。摘要保留原 sequence、handle、Task、producer、revision 和 hash，运行中的 Task、未解决错误和未合并修改不会被摘要移除。

3.3.4. 结构化查询历史与预测输入

成功只读文件查询还可提交机器可读签名，包括操作类型、tool id、稳定文件身份、revision、workspace object、查询指纹、offset、length、Context/cause sequence、tick、lifecycle、branch generation 和 Agent identity。内核为每个 Agent 保存 16 项固定转移表，依据 active path 上的 $A \rightarrow B$ 观察数与置信度产生低优先级预取。Guest VFS 目标进入异步预取队列，Host 目标产生带 object id、revision 和 range 的 PREFETCH_HINT。

训练只接受成功结算的只读记录。失败、取消、超时、拒绝、未完成 Task 和回滚分支不会进入转移表；Context 清空、Agent 退出、lifecycle、文件 incarnation、revision 或 workspace

generation 变化会使相关状态失效。预取只提升缓存命中率，正式读取仍执行 capability、文件访问范围、workspace root、revision 与 lifecycle 检查。

3.3.5. 数据来源传播

AgentOS 使用六类可组合的数据来源标签，标签随上下文、文件、工具返回、IPC 和 artifact 传播。

标签	含义	典型来源
KERNEL_FACT	内核生成的结构化事实	PID、资源、调度器、能力位
TRUSTED_USER_CONTROL	经产品界面提交的用户控制信息	新交互轮次、参数绑定审批
AGENT_DERIVED	智能体根据已有输入推导的数据	摘要、计划、报告
UNTRUSTED_FILE_DATA	来自文件的内容或元数据	查询、读取、摘要哈希
UNTRUSTED_TOOL_OUTPUT	工具执行返回	带类型信息的 V2 或 V3 结果
CROSS_AGENT_DATA	经 IPC 或 Artifact 在 Agent 间传递	任务结果、编排 Agent 接纳的公共 Artifact

表 3.4: 上下文的数据来源标签

文件、工具、邮箱和任务资源按保守的按位或规则传播来源标签。生成摘要、重新计算哈希和在智能体之间整理 Artifact 时会增加新来源，同时保留既有标签。文件数据保留 UNTRUSTED_FILE_DATA 来源，控制授权由能力位和 Execution Contract 独立判定。

3.3.6. 工具清单与副作用检查

每个内核工具都有安全清单，其中列出可接受的输入来源标签、输出新增标签、所需能力位，以及文件、元数据、IPC、进程、权限、artifact 和订阅的副作用掩码。启用 ENFORCE V3 后，生命周期代次、执行约定中冻结的边、schema、能力位、工具清单和数据来源必须同时匹配。失败请求会在工具产生副作用前返回结构化状态。只有成功预留并提交关键记录的拒绝才取得执行记录票号；空间不足时调用返回 NO_SPACE 或 RETRY。

3.3.7. 执行记录环

每个活动 workflow 按需分配四个智能体状态页：三页普通记录环，共 48 个 256 字节槽位；一页关键记录环，共 16 个槽位。直接调用或 V3 调用的标准完成或拒绝路径会预留递增的执行记录票号，填充后再提交；手工备注、回滚和清空不使用这套预留和提交规则。当较早编号仍在准备时，读取者会停在当前位置，从而保持全局发布顺序。

普通成功的上下文记录写入普通记录环，获准副作用和成功建立记录的拒绝写入关键记录环。拒绝记录无法预留或提交时，直接调用或 V3 调用返回 NO_SPACE 或 RETRY，关键记录票号保持为空。复用环形空间前，已提交分段会汇入内部根记录。该结构为 Workflow Fence 提供按执行记录票号排序的执行记录。

3.3.8. Workflow Fence

具备 workflow 控制权限的 Agent 通过 agent_run(count=0, AGENT_RUN_F_FENCE) 发起 Workflow Fence。内核检查调用者与控制标识的关系，使生命周期进入屏障状态，排空元数据、实时查询延迟工作和 VFS 回收，检查执行与存储账户中的待结算额度归零，最后密封执行记录环。

屏障请求携带版本号、`struct_size`、必须为零的 `flags` 与 `reserved`、挑战值和请求编号；前序根记录由内核在阶段快照完成后写入返回回执。请求和回执可以同时为 `NULL`，此时内核执行匿名且不等待结果的屏障。提供回执时，生成的 320 字节回执绑定前序根记录、挑战值、屏障序号、执行记录票号范围、事件与缺口计数、元数据代次、额度周期、精确已用额度摘要和有序分段摘要。请求编号非零且提供回执时，内核在内部状态提交后复制结果；同一个请求编号再次复制时返回缓存结果。该回执表示当前启动周期内的工作流阶段快照，其标志同时记录元数据是否易失以及覆盖状态。

3.3.9. 验证结果

`agentfinal_ucore` 连续执行 64 次工具调用，并覆盖快照、分支、回滚和窗口淘汰。数据来源场景把文件来源传入受保护的副作用，验证副作用前的拒绝与关键记录环记录。屏障场景覆盖挑战值、重试缓存、待结算额度、元数据代次和结果复制重试。

3.4. 面向 Agent 查询优化的文件系统扩展

3.4.1. 从路径访问到属性查询

科研和恢复工作流程经常需要回答两个问题：当前哪些对象满足条件，哪些对象刚刚发生变化。传统路径访问适合已知文件名的场景，却不能直接表达“查找某个阶段、状态或类别的结果文件”。AgentOS 在 uCore VFS 之上增加按启动周期隔离的 `Metadata Catalog`，为显式登记的文件维护项目、运行、阶段、状态、类别、依赖关系和内容摘要。Agent 可以据此按属性选择对象，并继续订阅对象变化。

3.4.2. 元数据登记与 inode 代次

拥有 `META_WRITE` 的智能体可以调用 `agent_file_meta_set()` 写入单条元数据，也可以通过 `agent_file_meta_set_batch()` 顺序提交一组登记。字段与真实 `dev+inum+incarnation` 绑定，更新通过 `update_mask` 完成，删除使用显式 `DELETE` 动作。文件创建和重命名仍沿用 uCore VFS，只有工作流明确关心的对象才会进入 `Metadata Catalog`。

批量接口每次最多接收 16 项，整个调用只进入一次工作流生命周期操作，并共享一次元数据事务。内核先在进程 VM 快照下复制全部输入，同时准备状态输出页；输入数组与状态数组需要使用互不重叠的地址区间。完成准备后，内核依次提交各项，并为每项写回独立状态。某项返回普通业务错误时，已经成功登记的前项继续保留，后项仍可处理；遇到无法确定提交结果的故障时，接口停止在已处理前缀并返回 `INDETERMINATE`。这种顺序提交规则使 Guest 能够从返回数量和逐项状态继续处理，无需再次登记已经确认成功的对象。

对象代次用于区分 `inode` 编号复用前后的文件。VFS 以增量方式维护代次，并随查询和订阅结果返回。查询或订阅命中记录后，内核会复核 `Metadata Catalog` 代次、生命周期、访问范围和完整条件。`unlink`、内容大小变化和对象失效都会更新已登记的附属记录。

3.4.3. 遍历查询与选择性索引

Metadata Catalog 为 status、stage 和 kind 维护选择性索引。带类型信息的查询只携带条件字段、查询标志和最大命中数；访问范围由调用者身份限定，查询计划由内核选择。结果返回命中数、候选记录数、扫描记录数、全部命中数、实际返回数、truncated、查询计划、计划原因和文件系统代次。

调用者可以用 SCAN 强制遍历，USE_INDEX 则允许内核使用索引。启用索引时，内核按固定的 status->stage->kind 顺序选择第一个已给定字段，再对候选记录检查完整条件，因此遍历与索引查询返回相同结果。最大返回数为 8，total_hits 仍报告全部命中数；当前 ABI 没有分页游标。查询结果是带属性、摘要、digest 和对象代次的定长结构，可以直接写入 Agent Context 区第七页的用户缓存。

3.4.4. 带类型信息的实时订阅与快速跳过

agent_live_watch() 安装一份完整的 agent_file_query。订阅器只保存查询条件、访问范围和代次，不保存持久结果集。每次元数据变化时，内核重新计算变更前是否满足条件：对象从未命中变为命中时产生 ENTER，对象持续命中且记录改变时产生 UPDATE，对象从命中变为未命中或被删除时产生 LEAVE。事件以 AGENT_EVENT_FILE_QUERY 写入目标智能体的 Context 和定长队列。

在早期开发中，每次元数据变化都会扫描进程表，确认是否存在匹配的文件订阅。多数 workflow 只执行查询，没有安装 Watch，这段固定检查会落在每次登记和更新路径上。当前实现为每个进程维护“是否存在文件订阅”的精确缓存，并在中断保护下汇总为全局进程计数。元数据变化完成对象与生命周期检查后，如果计数为零便直接结束事件发布；存在订阅时才进入进程扫描。安装、按 watch_id 移除和进程回收都会同步更新计数。通用文件状态订阅与 Typed Watch 使用各自的移除入口，清理前者不会删除仍在工作的 FILE_QUERY 订阅。

队列或待处理表无法保存全部增量时，内核会推进 resync_generation 并发送 RESYNC_REQUIRED。恢复时，智能体保留旧订阅，先以相同条件安装不带 ACK_RESYNC 的替换订阅，再执行定长查询。完整基线要求快照满足 truncated=0；随后在旧订阅上以对应代次执行 ACK_RESYNC 并取消旧订阅，替换订阅覆盖整个切换时段。单次查询最多返回 8 个命中项，快照截断时调用者需要收紧查询条件。

Workflow Fence 会排空删除标记、内容增量并尝试投递标记事件。workflow 或访问范围中的重新同步尚未由 ACK_RESYNC 确认时，屏障返回 RETRY；最终回收阶段再清理残留对象。

3.4.5. 从逐项建档到自适应复用

在早期开发中，综合恢复场景在每次查询前逐项登记 96 条 Metadata，完成一次索引查询后立即清理。索引缩小了查询阶段需要检查的记录数，96 次登记、重复建档和一次性清理仍会进入完整流程。配对测量由此出现了明显反差：核心查询已经稳定加速，完整流程中的准备与收尾时间却抵消了这部分收益。

当前实现先估计相同发现条件的预计使用次数 K 。当 $K = 1$ 时，场景直接遍历目录，不创建短寿命 Catalog；当 $K \geq 2$ 时，Guest 以每批 16 项登记对象，将 Catalog 从 BUILDING 推进到 READY，后续查询在同一 lifecycle 内复用已验证的索引。

对象发生恢复更新时，状态短暂进入 DIRTY，Metadata 更新完成后重新回到 READY。生命周期结束时才集中清理文件与登记记录。路径选择同时控制冷建档成本和重复查询成本，使一次性使用与多次使用分别走适合自己的执行方式。

3.4.6. 查询性能

目录规模与命中数对索引收益的共同影响如图3.1所示。实验使用目录规模 24、64、96 与精确命中数 1、2、4、8，组成 12 个实测网格；每个网格保留 15 个 AB/BA 内部配对。图 a 逐格给出“遍历耗时除以索引耗时”的中位数，图 b 将同一组中位数展开为参数曲面，用于观察索引收益随目录规模和命中数的变化。

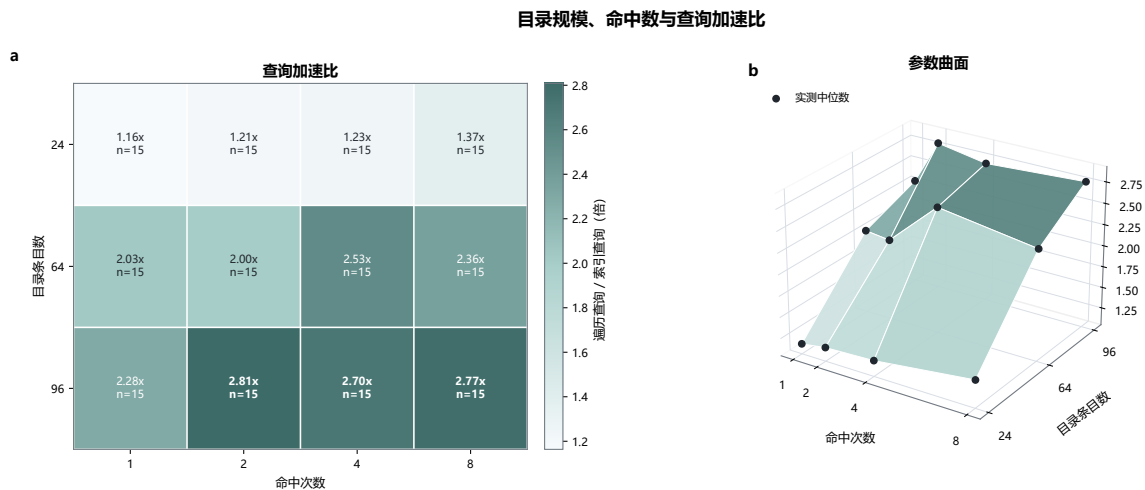


图 3.1: 目录规模、命中数与 indexed 查询加速比

图中的 12 个实测中位加速比为 1.164–2.808 倍。目录规模从 24 增至 64、96 时，indexed 更快的内部配对数分别为 43/60、60/60、58/60，收益整体随目录扩大而上升。固定的索引选择顺序先缩小候选集，再逐项复核完整条件；遍历路径检查的无关记录会随目录增多。24 项目录中仍有 17/60 个配对不占优，这一现象促使上层将预计使用次数纳入路径选择，避免为单次使用支付冷建档成本。

首轮遍历、后续遍历与索引查询在三种目录规模下的耗时分组如图3.2所示。横轴按目录条目数分组，每组的三根柱分别表示显式预热后的首个遍历样本、其余遍历样本和索引就绪后的查询样本；柱高为单次查询耗时中位数，误差线表示四分位区间。

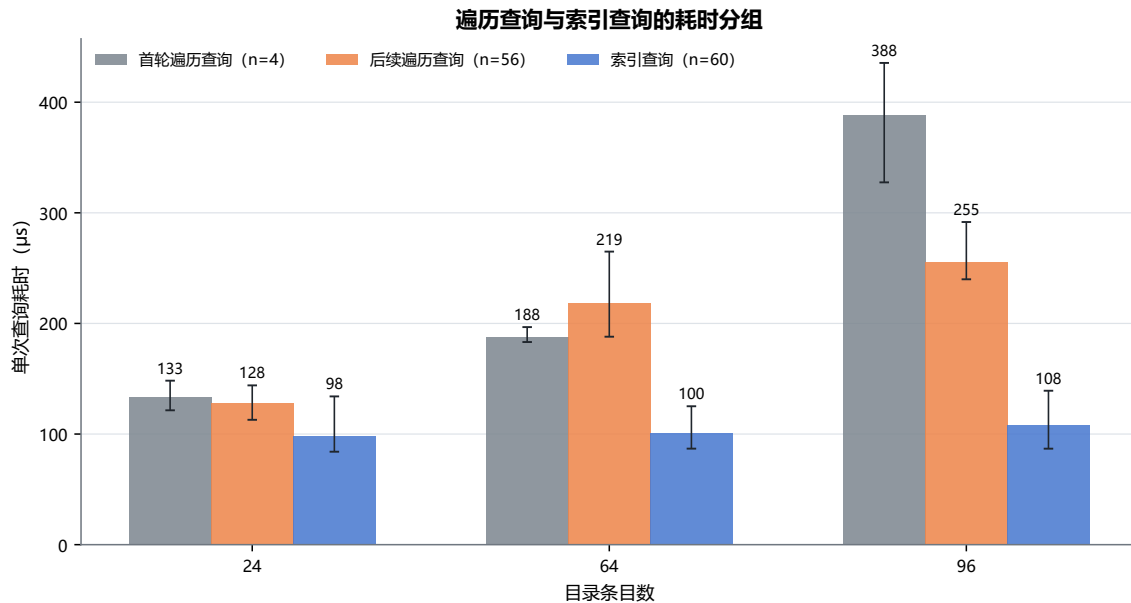


图 3.2: SCAN 与 indexed 查询的耗时分组

indexed 查询的中位耗时随目录规模由 98.3 微秒升至 108.3 微秒；在 96 项目录中，首轮 SCAN 和后续 SCAN 分别达到 388.5 微秒和 255.1 微秒。28 条诊断记录均为 `cache=ready`，图中的首轮 SCAN 表示预热后保留的首次完整遍历计时，不包含系统启动时间。索引查询在重复使用阶段保持稳定，完整遍历仍会受到目录检查和运行状态影响。

批量建档与同 lifecycle 复用后的完整流程结果见图 3.5。在 $K = 4$ 的 16 次独立 QEMU 启动中，96 条记录由 6 次批量调用完成登记，Catalog 只构建一次；4 次发现查询和 1 次恢复验证共复用 4 次。索引复用路径在核心窗口和完整流程窗口均为 16/16 个配对更快，配对中位差分别为 -105.668 ms 与 -89.782 ms 。遍历路径与索引复用路径的核心耗时中位数分别为 121.206 ms 和 19.500 ms ，完整流程中位数分别为 786.684 ms 和 697.807 ms 。结果说明批量登记消除了逐项系统调用的大部分固定开销，生命周期内复用又把一次冷建档分摊到多次查询。

3.4.7. 验证结果

`agentfs_ucore` 覆盖单条与批量元数据设置、更新、删除、摘要、对象代次和访问范围，并检查混合逐项状态、失败前缀以及不跨项回滚。`agentscan_ucore` 覆盖 ENTER、UPDATE、LEAVE、队列缺口、RESYNC_REQUIRED、替换订阅和零订阅快速路径，还会确认通用清理保留 Typed Watch。综合场景进一步核对 $K = 1$ 的遍历选择、 $K \geq 2$ 的批量建档、同 lifecycle Catalog 复用以及两条路径相同的恢复结果。

3.5. Agent Loop 内核运行机制

3.5.1. 从轮询循环到内核等待

Agent Loop 会等待用户输入、模型回复、文件变化和其他 Agent 的消息。持续轮询不仅频繁进入内核，还会让大量等待线程占用调度时间。AgentOS 使用有界事件队列、订阅和等待接口，让没有事件的线程真正睡眠；消息、文件事件或 TIMER 到达后，内核再把相应线程唤醒。

每个智能体拥有容量为 16 的事件队列，其中保留一部分内核槽位，同一事件来源最多排队 4 条事件。路由表同样具有固定容量。发送 MESSAGE 时，内核检查 MESSAGE_SEND 能力位、活动工作流、发送者、目标路由和订阅状态。公开的 agent_event 只返回发送者、目标 PID、关联号等事件字段；control_id 和数据来源标签保存在内核附属记录中。

Agent 使用 agent_watch() 登记关心的事件，以 agent_wait() 休眠等待，再用 agent_unwatch() 取消订阅。心跳设置与停止接口负责调整定时事件。Loop 的下一步决策仍在 Guest 中完成，内核只负责保存订阅、管理等待状态，并在事件或 TIMER 到达时提供可靠唤醒。

非 TIMEOUT 的 agent_wait 会先检查已排队事件。选中事件时，内核建立保留项；没有事件时，内核以线程身份代次作为等待键安装等待者并让线程睡眠。事件入队和等待者状态使用相同锁序，确保到达事件与等待代次对应。TIMEOUT 路径在本地直接返回，不建立保留项，也不写入上下文。agent_wait_cancel() 仅向等待路径注入取消事件，工具和任务取消由各自协议处理；消费事件后才更新循环状态和上下文。

线程进入工具执行、等待事件和结束本轮处理时，内核分别维护 RUNNING、WAITING 和 IDLE 状态，并把同一进程中各线程的状态汇总到 PCB。agent_info() 可读取这一状态，观察程序也会把它与唤醒和调度记录对应起来。任务结束后，Guest 沿普通 exit 路径退出；生命周期在成员、在途操作和资源引用排空后统一回收 Context、订阅与队列对象。

3.5.2. 心跳与模型请求关联号

Agent 可以设置心跳间隔。定时器到期时，内核生成 TIMER 事件并唤醒 Agent；Guest 再根据当前状态决定检查文件、发送消息或继续等待。停止心跳会阻止后续 TIMER 产生，但已经进入事件队列的 TIMER 仍按普通事件消费，避免悄然改写队列历史。

LLM_REQUEST 与 LLM_RESPONSE 复用消息与等待机制。请求会登记生命周期、请求者和中继程序的 PID、控制标识、关联号和截止时间。同一请求者的关联号严格递增，并且只有成功入队后才更新。中继程序返回匹配响应时，内核原子消费待处理请求并生成 LLM_DONE。只有同一关联号仍处于活动待处理状态时才返回 DUPLICATE；复用已经完成或过期的关联号则返回 CONFLICT。待处理请求使用 120 秒内核时钟 TTL，完成历史区分已消费请求和超时请求。

3.5.3. 工作流资源账户

资源账户维护已用额度 U 、待结算额度 P 、空余额度 F 和总额 $U + P + F$ 。当前账户的可用额度先在本地图态中转移：

reserve:F→P commit:P→U cancel:P→F release:U→F

账户补充额度时会检查全局限制、资源类别和账户硬限制。系统压力、上下文切换、账户关闭和 Workflow Fence 都会收缩 F 。Workflow Fence 在同一把锁内执行 trim+snapshot，要求 $P = 0$ ，并导出精确 U 、账户代次、额度周期和向量摘要。

3.5.4. 按 workflow 汇总的 EEVDF

AgentOS-uCore 将 EEVDF 的候选实体从线程提升为 workflow，使同一资源账户中的线程共享服务份额。实现中，调度器使用 workflow 虚拟运行时间 (vruntime) 计算滞后量，满足 $vruntime \leq global_vtime$ 的实体进入可服务集合。延迟类别、事件和截止时间可以缩短服务请求。一次分派完成后，内核根据实际服务周期更新 workflow 虚拟运行时间；睡眠实体的滞后量会向全局虚拟时间衰减。

只有一个 workflow 实体时，调度器使用快速路径。实体表容量为 4，其中包含一个 BOOT_SEALED 启动参与者，因此最多可同时纳入三个新建 workflow。实体信息不完整或容量超出时，调度器回到原 uCore 路径，并增加回退计数。

3.5.5. 唤醒延迟与公平性

唤醒与公平性测量使用 6 次全新 QEMU 启动，覆盖并发度为 1、2、3、4 的服务窗口，并为并发到达和线程放大场景保留精确唤醒探针。探针直接读取调度器跟踪中的 ready_age，公平性则由同一启动、同一并发组内各 workflow 的 service_cycles 计算。

workflow EEVDF 的唤醒等待时间与调度公平性如图 3.3 所示。图 a 给出五种负载下唤醒延迟的累积分布，横轴为等待的 tick 数，纵轴表示在该 tick 数以内获得服务的探针比例；图 b 给出 1-4 个并发 workflow 在各次启动中的 Jain 公平指数，散点为单次启动结果，折线和阴影分别表示中位数与四分位区间。左图观察事件到达后能否及时运行，右图观察 CPU 服务量能否按 workflow 接近均分。

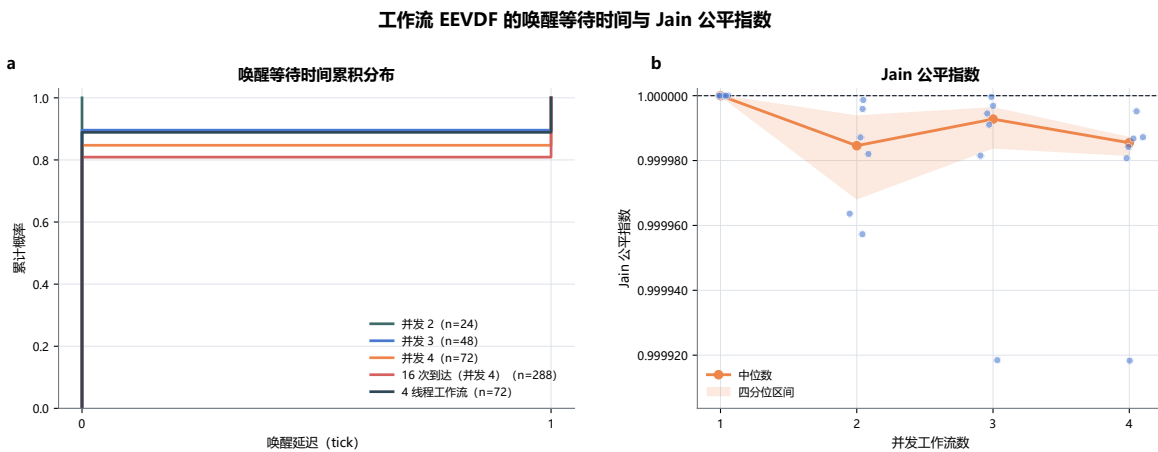


图 3.3: 工作流 EEVDF 的唤醒 tick 与调度公平性

图 a 的 504 条精确探针中，425 条在 0 tick 获得服务，79 条等待 1 个 tick，五条曲线都在 1 tick 处达到 1。当前实验使用单 Hart，uCore 的 tick 为 10 ms，因此所有被唤醒的工作流都在一个调度节拍内进入运行。图 b 中，并发度为 1、2、3、4 时，Jain 指数中位数分别为 1.000000、0.999985、0.999993 和 0.999985，各组散点也集中在接近 1 的区域。

这一结果来自 workflow 级记账：调度器把同一 workflow 内的线程折叠为一个候选实体，再按 workflow 虚拟运行时间分配服务，从而避免线程数量直接放大 CPU 份额。在独立的四线

程放大场景中，Jain 中位数为 0.9980725，该 workflow 取得 26.895% 的服务份额，说明聚合记账已经显著抑制倾斜，但多线程 workflow 仍会带来小幅偏差。因此当前设计继续保留 workflow EEVDF 作为外层调度单位，并将在更细粒度计时、更多实体和 SMP 环境中复核唤醒和服务分配。现有结论对应单 Hart、10 ms tick 和最多四个实体的负载。

3.5.6. 验证结果

agentloop_ucore 覆盖原子等待、唤醒、超时、取消、心跳和消息路由；调度场景覆盖快速路径、实体容量、线程放大、睡眠衰减、回退、唤醒直方图和 Jain 公平性。

3.6. AgentOS 控制台综合 workflow

3.6.1. 科研恢复场景

AgentOS 控制台用六组内核机制执行一个确定性的科研恢复 workflow。labdemo_ucore 创建协调、哨兵、调查和恢复四个角色，建立 workflow 身份、MESSAGE 路由、上下文、元数据和资源账户。场景登记 prepare、align、analyze、report、archive 五个阶段及其依赖关系，再查询失败节点、读取摘要、向其他智能体发送恢复请求，完成 align 恢复和 report 更新。

固定策略为每个阶段指定相同的业务工作量，工具结果仍由 AgentOS 内核执行和检查。因此，该场景同时覆盖身份、上下文、实时查询、IPC 等待和恢复写入，并能在不同实现路径之间进行等量比较。

3.6.2. 遍历与索引路径的配对实验

综合场景保留遍历实现和索引复用实现，两者的恢复状态和结果哈希相同。每次独立赛题平台启动按 A/B 或 B/A 顺序运行，核心窗口覆盖查询、恢复写入、fsync 和复核，查询区段单独记录纯查询时间。

正式 campaign 使用 16 次独立 workflow 启动，每次设置 $K = 4$ 次发现查询，A/B 与 B/A 各占 8 组。两种执行顺序交替出现，使同次启动内的配对共享文件系统和工作负载状态，同时避免同一条路径持续先运行。

如图 3.4 所示，遍历查询与索引复用的核心耗时由三个面板共同呈现。图 a 用横线连接每次赛题平台启动的两个结果，用于检查收益是否在每个配对中保持同一方向；图 b 将 16 组原始耗时汇总为散点、箱线和半侧密度，并保留灰色配对连线；图 c 给出“索引复用减遍历查询”的累积分布，黑色虚线为零，橙色点线为配对差值中位数，落在零左侧表示索引复用更快。

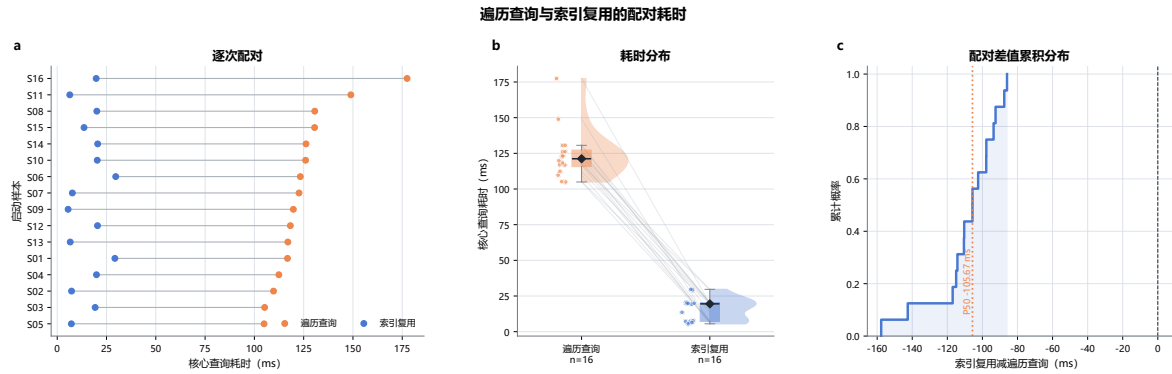


图 3.4: 遍历查询与索引复用的配对耗时

图 a 的 16 条配对线均连接到耗时更低的索引复用端点，图 c 的差值分布也位于零左侧。遍历查询和索引复用的核心中位数分别为 121.206 ms 与 19.500 ms，中位耗时之比为 6.216 倍；16/16 个配对均由索引复用路径更快完成，配对中位差为 -105.668 ms。选择性索引先减少候选记录，再执行与遍历路径相同的条件复核，收益来自跳过与筛选条件无关的目录记录。

8/8 的顺序均衡使每条路径都能在先运行和后运行的条件下接受比较。性能回归采用同次启动配对并轮换 A/B 与 B/A 顺序，以核心窗口的配对差值为主要判据。下节说明批量登记与 Catalog 复用如何把核心收益保留到完整流程。

3.6.3. 核心查询与完整流程窗口

核心窗口用于观察 AgentOS 查询路径，计时范围包含查询、恢复写入、fsync 和结果复核；完整流程窗口还包含文件准备、必要的元数据建档和查询结束后的文件清理。两个窗口均在同一次启动内配对，因而可以区分查询优化本身与整个场景的总耗时。

优化前的同类工作负载中，索引查询已经缩短核心段，但 96 条元数据逐项登记使完整流程仅有 3/16 组更快，配对中位差为 $+13.452$ ms。随后，工作负载把 96 条记录集中为 6 次批量调用，Catalog 只冷建 1 次并在后续 4 次查询中复用；检查记录数由 385 降至 5，降幅为 77 倍。

如图3.5所示，批量登记与 Catalog 复用后的核心查询和完整流程配对结果由三个面板展示。图 a 连接 16 次启动中的核心窗口结果，图 b 连接相同配对的完整流程结果，黑色菱形连线表示各路径的中位数；图 c 把两个窗口的“索引复用减遍历查询”差值置于同一坐标轴，零左侧表示索引复用更快。这样的并列方式可以直接观察优化后的收益能否延续到完整流程。

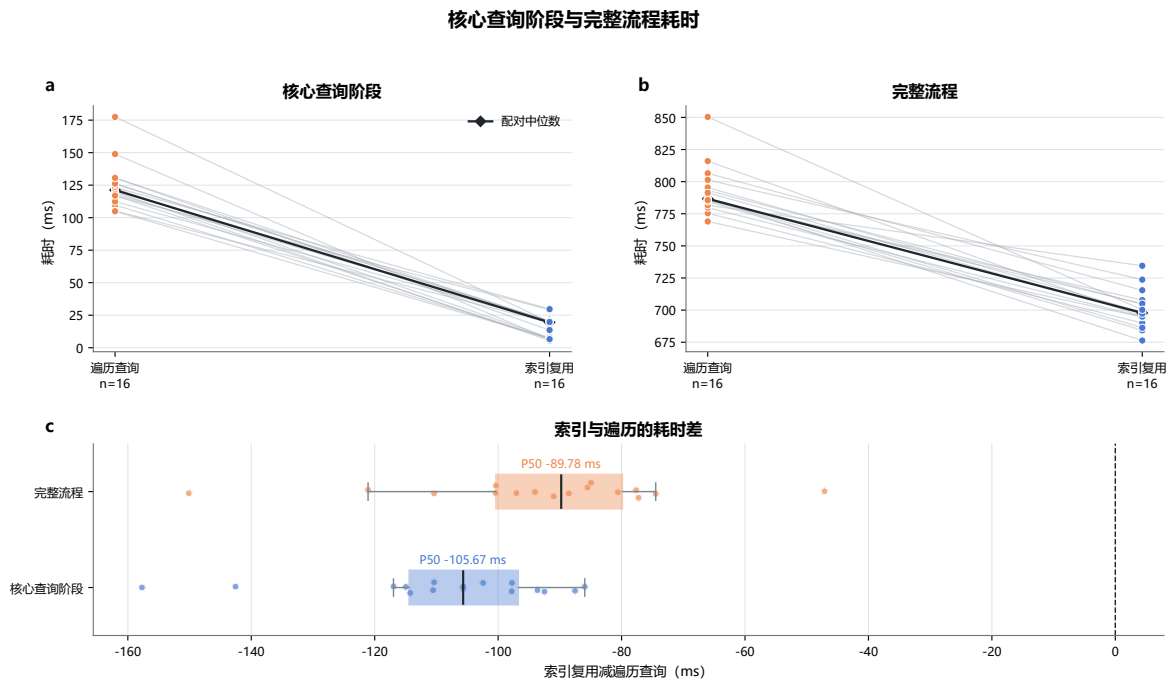


图 3.5: 核心查询阶段与完整流程耗时

图 a 中，核心段遍历查询和索引复用的中位数分别为 121.206 ms 与 19.500 ms，配对中位差为 -105.668 ms，16/16 个配对均更快。图 b 中，完整流程的中位数由 786.684 ms 降至 697.807 ms，中位耗时之比为 1.127 倍，配对中位差为 -89.782 ms，16/16 个配对同样更快。图 c 的两组差值均在零左侧；两条路径的结果哈希完全一致，说明时间改进没有改变恢复结果。

本次冷建的中位耗时为 11.172 ms，包含发现与复核在内的聚合查询耗时为 3.495 ms，热复用查询为 515 微秒。结果表明，批量登记消除了逐项进入内核的固定成本，Catalog 复用让同一 workflows 内的多次发现查询共享已经准备好的索引；性能分析继续保留核心窗口与完整流程两个视角。

3.6.4. 系统结果

综合工作流依次执行智能体创建、上下文记录、带类型信息的工具调用、实时查询、消息等待、恢复写入和执行记录。输出包含业务指纹、各阶段状态、检查记录数、核心与完整流程时间以及工作量计数。遍历查询和索引复用都在同一 AgentOS-uCore 内核与同一 Guest 场景中运行，并以相同的用户态约定比较两种查询实现。

3.7. 综合场景：AgentOS Nexus 通用多 Agent Harness

3.7.1. 共同 Agent Loop 与动态配置

AgentOS Nexus 是构建在 AgentOS-uCore 之上的通用多 Agent Harness。CLI 只接收用户目标、workspace root、workflow policy、资源限制、允许工具和可选 Agent 配置，不为某类应用预设固定流程。所有 Agent 使用相同运行时、Context、Task Channel 和工具协议；名称只用于日志。差异来自 capability、允许工具、资源额度、系统提示和任务描述。拥有 ORCHESTRATE capability 的 Agent 可以承担编排职责，并动态决定子任务、依赖、并行度和汇总时点。

示例配置	可承担工作	主要授权
编排配置	分解目标、创建子 Agent、建立 Task、接纳结果	ORCHESTRATE 与 policy 允许的工具子集
只读配置	搜索工作区、读取文件、观察 Guest、形成报告	内容读取、系统观察、Artifact 读取与发布
写入配置	根据 revision 写文件或应用补丁	工作区写 capability、受控路径和原子提交
构建运行配置	隔离构建并在独立 Guest 中执行测试	固定目标、工具链、build id、时间与资源额度

表 3.5: 同一 Agent Loop 的可配置能力组合

用户策略给出 workflow 可使用的最大 capability、工具和额度。父 Agent 只能授予该集合与自身权限的交集，多个 Agent 也可以使用完全相同的配置。Host Runtime 持有传输配置、服务接口密钥和模型格式适配器；workspace、build 与 run broker 只执行 descriptor 已授权的受控操作。整个会话启动一个长期运行的 agentharness_ucore Guest，内核负责身份、权限、Task、Artifact metadata、Context、资源、调度和 lifecycle。

3.7.2. 一次交互回合

Nexus 一轮交互中的运行时、内核与受控 Provider 协作如图 3.6 所示。目标与 workflow policy 进入共同 Agent Loop，运行时按配置建立 Agent 实例；Host 每创建一个 Agent，长期 Guest 就通过 agent_runtime_control (SPAWN) 创建对应受控进程。AgentOS 内核维护动态 Task 与终态交付，workflow 共享索引汇总已经结算的 Artifact、revision、build 与测试；受控 Provider 执行工作区读写、隔离构建和独立 Guest 运行。

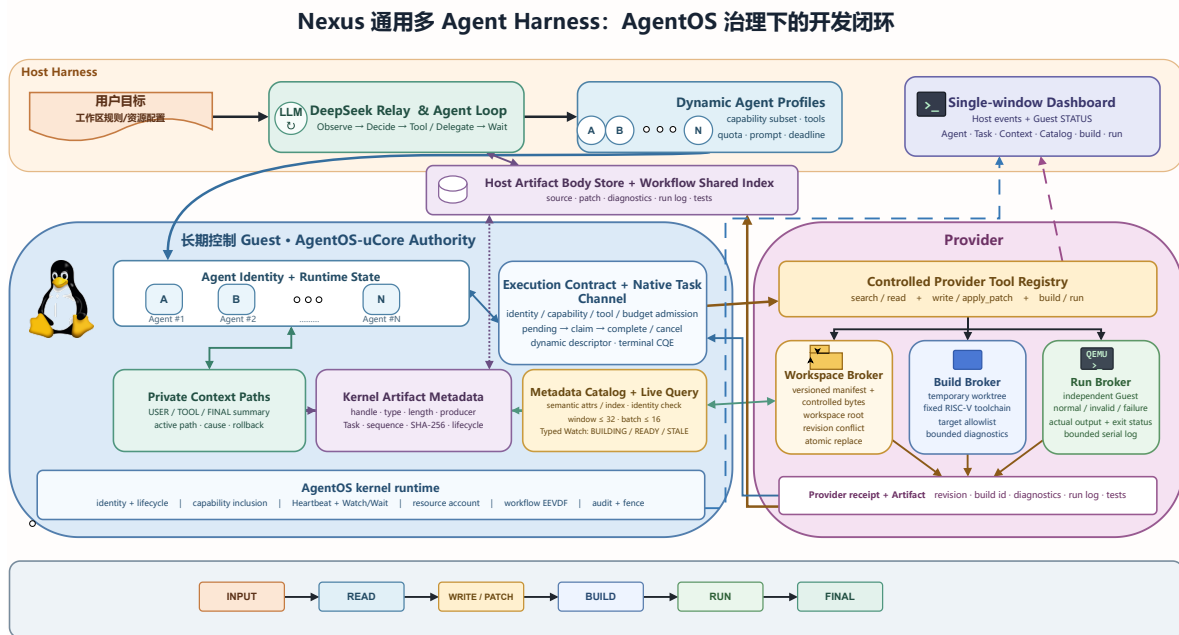


图 3.6: Nexus 通用 Agent Loop、动态 Task、Artifact 与预测性预取

图中的一次完整往返可按以下顺序理解。

1. 用户目标写成 root Task 的目标 Artifact，根 Agent 的 private Context 追加 USER 摘要；模型可直接完成，也可申请工具或建立子任务。
2. 需要子任务时，父 Agent 选择目标配置，并在 128 字节 descriptor 中绑定 parent task、目标与输入 Artifact、capability、工具、workspace revision、预算、deadline 和结果类型。
3. Host 把 root Task 与子 Task 送入同一 Guest 的原生 Task Channel。内核检查父子身份、lifecycle、授权包含关系和 Task graph，再将请求推进到 pending；目标 Agent claim 后继续同一套模型循环。
4. 工作区、构建或运行请求交给相应 broker。文件操作核对 root 与 revision，构建使用临时工作树和固定工具链，运行用独立 AgentOS-uCore Guest 取得真实输出。
5. 子 Agent 先封存结果 Artifact，再提交 terminal 状态。父 Agent 从 CQE 取得 handle 后复核 producer、Task、来源 sequence、lifecycle 和 SHA-256，随后才接纳结果。
6. 成功结果进入 private Context；明确共享的 Artifact 同时进入 workflow 索引。达到高水位时生成保留 Task、revision、build、测试和未解决问题的结构化摘要。
7. 已结算的只读查询可训练固定容量转移表；置信度满足阈值后，Guest 预取队列或 Host PREFETCH_HINT 提前载入下一项，正式读取仍再次检查权限与 revision。

3.7.3. 动态 delegated Task Channel

内核工具目录中的第 26 项用于任务委派，接收准确的 128 字节 AGENT_ARTIFACT_TASK descriptor。描述符绑定目标 identity、parent task、目标描述 Artifact、输入 Artifact、所需 capability、允许工具、workspace revision、资源预算、deadline 和预期结果类型。大段正文留在 Context Artifact Store，CQE 只返回状态、handle 与关联信息。

父 Agent 是 Task Channel owner。目标必须持有任务接收 capability，并获得与普通消息分离的 TASK route。内核分别检查用户 workflow policy、父 Agent 权限与子 Agent 配置，拒绝 self delegation 和 Task graph 中的真实环路，同时允许一个父任务并行维护多个独立子任务。提交成功后任务进入 Execution Contract 的 RUNNING 状态和 pending 队列；目标使用 claim/complete 系统调用领取并提交终态。任务完成终态交付时，owner 至多收到一条 terminal CQE。相关 ABI 名称和系统调用号如下：

```
1 capability  AGENT_CAP_TASK_ACCEPT
2 route       AGENT_IPC_ROUTE_TASK
3 syscall 567 agent_task_delegate_claim
4 syscall 568 agent_task_delegate_complete
```

claim 同时建立 delegated effect lease，精确绑定 worker 身份、线程 generation，以及通道、请求和槽位代次。worker 或已登记 helper 的直接副作用必须是 Contract manifest 的子集，complete 以及终态确认都要等待活动 lease 引用归零。子 Agent 先封存结果 Artifact，再提交 terminal 状态；父 Agent 收到 CQE 后重新核对 producer、Task、Context sequence、lifecycle 与内容 hash，复核成功后才把结果并入自己的 Context。

同一活动生命周期内具备编排权限的 Agent 通过上述 complete 接口的取消标志发起请求：它提交 CANCELLED、零 ACK 字段以及准确的 owner/channel/request/slot/task/correlation tuple；内核还要求 AGENT_CAP_ORCHESTRATE、AGENT_CAP_WAIT_CANCEL 和指向 owner 的活动 AGENT_IPC_ROUTE_TASK 路由。返回 OK 表示取消请求已被接收，任务终态仍以 terminal CQE 为准。若任务已经处于 CLAIMED，provider 须清理预绑定输出，再以 ACK_TERMINAL 确认最新 generation 的 offer。当前委派协议采用协作式终止，无法强制终止永久无响应的已领取任务。

3.7.4. 版本化工作区与受控 Broker

Host workspace broker 在会话开始时固定一个显式 root，并以 object id、相对 path、revision、大小和类型描述文件。revision 来自与后续搜索、读取或写入相同的已验证字节；broker 拒绝绝对路径、目录跳转、符号链接逃逸以及位于 root 之外的对象。search_files 返回有界候选，read_file 必须提交对应的 object/path/revision 绑定，结果正文封存为 FILE 或 SEARCH Artifact 后进入模型 Context。

写入工具继续沿用相同对象身份。write_file 与 apply_patch 在提交前核对预期 revision；发现其他 Agent 已经修改文件时返回冲突，调用 Agent 必须重新读取。提交过程先在目标目录创建临时文件，写完并同步后执行原子替换，再同步目录。工作区 Catalog 的批量登记、Typed Watch 和生命周期内复用已经进入通用 Harness 的搜索与读取路径；写入成功会使旧窗口进入 STALE，下一次查询从 cursor 0 重建并重新核对 generation、object id 与 revision。固定角色 Nexus 与旧 manifest 串口状态机不再进入产品路径。

3.7.5. 受控开发与真实 Guest 验证

write_file 和 apply_patch 只接受匹配 user/src/nexus*_ucore.c 的路径。创建文件时预期 revision 必须为 missing，修改现有文件时必须提交当前内容的 SHA-256；不匹配会返回冲突且不改动文件。写入先落到目标目录中的临时文件，完成数据同步后执行原子替换，再同步目录。补丁还要满足单文件 unified diff 的路径、hunk 与上下文核验。

`build_ucore_program` 把需要的源码、内核、用户库、文件系统和脚本复制到会话私有临时工作树。目标必须等于源文件名去掉 `.c` 后的值，工具链固定为 `riscv64-linux-gnu-`。构建过程限制为 180 秒、2 GiB 地址空间、256 MiB 文件、128 个进程，并把返回诊断控制在 2,200 字节内；成功时用源码、内核和镜像摘要生成 `build id`。

`run_ucore_program` 只接受当前会话内已经成功的 `build id`。每个用例复制一份测试镜像，以单 Hart 和 128 MiB 内存启动新的 AgentOS-uCore Guest，串口输入最多 512 字节，30 秒内收集程序实际输出、退出状态与日志。`case kind` 必须取 `normal`、`invalid` 或 `failure`。系统提示要求模型根据程序自己制定正常输入、异常输入和关键失败路径，开发完成时同时要求这些结果绑定最新 `source revision` 和同一 `build id`；后续写入会清空已有 `build` 与运行证据。

3.7.6. Context Artifact Store 与长任务摘要

用户态 Context Artifact Store 保存完整 `USER`、`TOOL`、`FINAL`、文件、搜索结果、补丁、编译诊断、运行日志、测试输出、子任务报告和团队摘要。单项正文最大 64 KiB，`workflow` 默认总额为 128 项和 2 MiB，模型单次投影不超过 12 KiB。内核 `syscall 571` 核验类型、长度、UTF-8、所有者和 SHA-256，并封存 `handle`、`producer`、`Task`、来源 Context sequence 与 `lifecycle`；封存完成后才允许 Context 或 Task 引用。

每个 Agent 保持 `private Context active path`，`workflow` 共享索引只引用已经成功结算且明确共享的 Artifact。`private Context` 达到高水位时，Nexus 生成包含目标、完成工作、工具、`revision`、`build`、测试、错误、待办和计划的结构化摘要；`team summary` 保存 Task graph、Agent 配置、依赖、公共 Artifact、冲突和未完成 Task。摘要保留原 `sequence`、`handle`、`Task`、`producer`、`revision` 和 `hash`。

`rollback` 只移动当前 Agent 的 `active path`。当前分支独占且失去引用的 Artifact 按引用计数和 `lifecycle` 延迟回收；已经被父任务接纳、被其他 Agent 引用或已修改共享工作区的结果继续保留。文件冲突由 `revision` 检查、原子替换和补偿 Task 处理。运行中的 Task、未解决错误、未合并修改和其他 Agent 正在依赖的 Artifact 不会被摘要清除。

3.7.7. 查询预测与受控预取

成功只读查询还会提交机器可读签名，包括操作类型、`tool id`、稳定文件 `identity`、`revision`、`workspace object`、查询指纹、`offset`、`length`、Context/cause sequence、`tick`、`lifecycle`、`branch generation` 和 Agent `identity`。内核为每个 Agent 保存 16 项私有转移表，依据 `active path` 上 $A \rightarrow B$ 的观察数与置信度产生低优先级请求；`workflow` 共享训练只接受明确共享且所有目标 Agent 都有读取权限的记录。

Guest VFS 目标进入异步预取队列，Host workspace 目标产生带 `object id`、`revision` 与 `range` 的 `PREFETCH_HINT`。默认单次最多 4 KiB、同时最多 2 项。失败、取消、超时、拒绝、未完成 Task 和回滚分支不进入训练；Context clear、退出、`lifecycle`、`incarnation`、`revision` 或 `workspace generation` 变化使相关状态失效。正式读取仍执行 `capability`、文件访问范围、`workspace root`、`revision` 和 `lifecycle` 检查。

3.7.8. 工具发现与内核观测

Nexus 启动时读取 AgentOS 内核的 33 项 Tool Registry，检查名称、编号和 schema，再按 Agent capability 与允许工具位图筛选。模型使用 search_files 与 read_file 读取工作区，使用 write_file 与 apply_patch 修改文件，随后可以选择 build_ucore_program 和 run_ucore_program。协作与结束分别使用 delegate_task 和 complete_task。同步 STATUS 负责 Guest 状态观察，不占用模型工具轮次。7 项 Host 工具各有独立 Registry 编号与 brokered 标志；普通调用得到 BROKER_REQUIRED，Harness 再把已授权请求交给受控 broker。模型无需构造二进制 descriptor，Harness 会把动作转换为绑定 parent Task、Artifact、capability、工具位、revision、预算和 deadline 的动态 Task。

观察路径通过 timeline、audit、identity、Context、Task Channel 和 Catalog 状态生成结构化事件。它可以显示 TASK route、claim/complete、terminal CQE、artifact 接收、Context sequence、Catalog generation、Typed Watch 失效以及 Agent 的等待和调度状态。任务推进与终态裁决由 Guest 和内核维护；Host 的有界 ledger 接收已结算投影和非正文关联信息，用于核对生命周期、父子任务和结果关联。

3.7.9. 运行验证

Host 开发回放固定写入、失败构建、修补、成功构建与三类 Guest 结果，确认后续源码修改会使旧 build 和用例证据失效。原生联合回归另行启动一个长期运行 Guest，把 Host Agent 映射为 Guest runtime 进程，并提交 root Task 与嵌套子 Task；claim、complete 和 terminal CQE 都由本次 Guest 生命周期产生。

在线 DeepSeek 计算器任务完成了通用开发闭环。模型自行选择单 Agent 方案，在 5 个模型轮次中依次调用读取、写入、构建和运行，创建 nexus_exprcalc_ucore.c，随后在固定 RISC-V 工具链中构建，并用同一个 build id 启动 5 个独立 Guest。正常表达式、非法字符、空输入、运算符错误和除零路径全部取得预期输出与退出状态。4 次产品工具调用分别形成 native Task，结果封存为 Artifact 并进入 Context；完成门在这些证据均绑定最新源码 revision 后才接纳 FINAL。运行时没有计算器名称、固定 Agent 数量或固定工具顺序的专用分支。

当前通用 Harness 通过 agentos_native_task_channel.py 把每次模型 Task 接入同一个长期运行 Guest。agentos-harness-native-test 在两次独立启动中验证 2 个配置 Agent、2 个嵌套 Task 与通用 Loop；agenttask_ucore 与 agentmulti_ucore 分别补充多子任务图、取消、Artifact metadata 和查询预测测试。固定角色与早期 Nexus 命令已经下线，旧命令会明确拒绝并引导使用通用 Harness。

3.8. 实现演进与核心片段

本节沿七条演进线说明系统如何从初赛阶段的简化设计收敛到当前实现。每条演进线依次回顾早期方案、分析功能组合后出现的问题，再结合前后对照图、核心代码片段和回归结果说明重构方法与当前效果。

3.8.1. 身份代次：从访问范围到生命周期键

在初赛的设计中，内核主要用单调递增的 `agent_id` 区分 **Agent**。随着文件访问和控制操作接入，VFS 又保存 `scope_id`，控制器和退出路径则分别记录 `control_id`。这些编号能够支撑简单的创建、通信和回收流程，但一次工作流的身份被拆散在多个模块中，异步引用缺少一把共同的核验键。

随着 VFS、IPC、Context 和资源回收开始共同引用一轮工作流，分散编号带来的问题逐渐显现：某个模块完成了退出，另一个模块仍可能保存旧任务或旧订阅；后续若把有界槽位交给新的运行，仅凭槽位编号也无法识别引用属于哪一次使用。继续在各模块增加局部状态，会让创建、撤销与回收分别形成一套判断规则。

经过这一分析，我们将槽位编号与复用次数合并为 `id+generation`，并在内核中拆出统一的生命周期账本。`id` 负责定位固定槽位，`generation` 负责识别该槽位当前承载的运行；任务回执、异步引用和回收屏障均保存同一把键。系统还评估过用分段持久租约维持跨重启连续性的方案，复核后将承诺限定为单次 boot 内唯一，删除磁盘恢复链路，并规定计数器不回绕、耗尽后关闭接纳。这样可以用一套可核验的规则覆盖创建、引用、撤销与回收全过程。

身份认定从分散编号到生命周期键的演进如图3.7所示。图中并列呈现初赛方案与当前方案，重点说明不同模块如何共享同一轮运行的身份，以及槽位再次使用后如何拒绝旧引用。

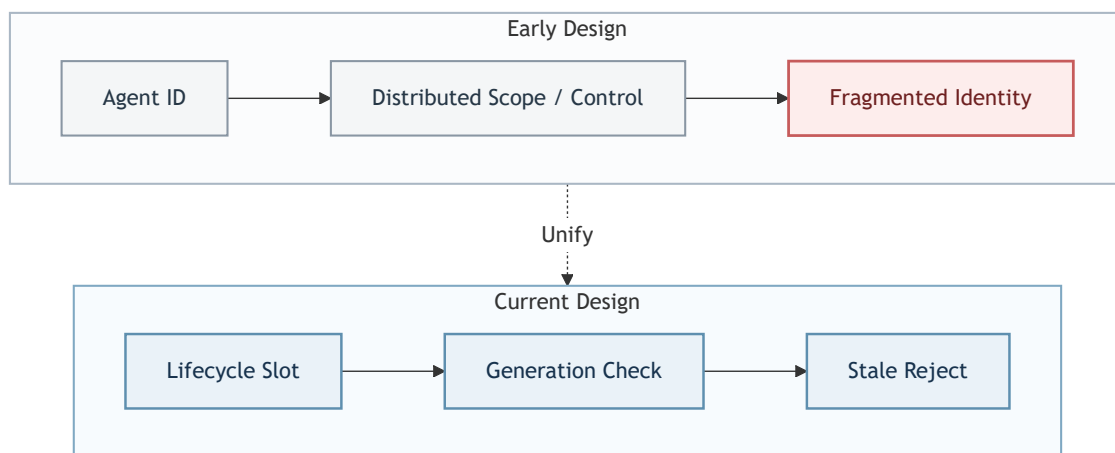


图 3.7: 身份认定由分散编号收敛为带代次的生命周期键

图中两条路径都自左向右推进。上方初赛路径从 `Agent ID` 进入分散的 `Scope / Control` IDs，最后形成 `Fragmented Identity`，表示 VFS、控制器和回收路径需要各自解释身份。下方当前路径从 `Lifecycle Slot` 进入 `Generation Check`：`id` 定位固定槽位，`generation` 识别

该槽位当前承载的运行；代次不匹配时立即执行 **Stale Reject**。统一的生命周期键使异步引用、资源记账和回收屏障依据同一条事实作出判断。

代码 1：生命周期槽位与代次的联合核验

c

```
57 static struct workflow_lifecycle_record *
58 workflow_lifecycle_find_locked(struct workflow_lifecycle_key key)
59 {
60     uint slot;
61     struct workflow_lifecycle_record *record;
62
63     if (!workflow_lifecycle_key_valid(key) ||
64         key.id > WORKFLOW_LIFECYCLE_CAP)
65         return 0;
66     slot = key.id - 1;
67     record = &workflow_lifecycles[slot];
68     if (!record->used || record->generation != key.generation)
69         return 0;
70     return record;
```

回归测试会主动复用相同编号，检查新代次前进，并确认旧键返回 **STALE**，从而锁定“旧引用不得复活”这一生命周期约束。

3.8.2. 结构化工具：从字段校验到 Execution Contract

在初赛的设计中，工具请求使用固定的 `arg0/arg1/payload` 三个值槽，并携带相应的 `key` 和 `type`。这一阶段的系统调用路径主要复制值槽，随后才加入逐工具的手写 `switch` 校验；工具表中的说明和真正执行的检查仍由两份代码维护。工具数量增加后，一次改名也可能暴露结构问题：旧键名 `response_summary` 恰好占满 16 字节字段，无法再容纳结尾 `NUL`。继续扩展手写分支会增加规则遗漏和两份定义失配的概率。

为解决字段规则分散的问题，我们将键名调整为可完整终止的 `reply_summary`，增加编译期容量检查，并引入有界 `typed key-value` 与 **Registry Schema**，统一核对名称、类型、长度、重复键和必填项。**V1/V2** 解码与工具列表共用参数规则；规则表随后压缩为连续数组和 `offset`，使无参数工具不再占据空槽。这一重构解决了“这组参数是否属于该工具”的问题，执行身份仍只由 `tool id` 和本次参数表示。

当 workflow 继续加入节点依赖、失败重试和截止时间后，相同工具与相同参数可能对应不同节点或不同尝试。接纳时通过校验的请求，也可能在真正产生副作用前已经取消、超时，或者失去其依赖的 **Context**。针对这一问题，当前 **V3 Execution Contract** 使用 **Contract key**、**node**、**attempt**、**schema/input fingerprint**、前驱 **Context**、**provenance**、**deadline** 和资源额度共同冻结一次执行计划，并在接纳阶段与副作用发生前分别核验。

结构化工具从字段校验到 **Execution Contract** 的演进如图 3.8 所示。当前方案将一次调用拆为 **Admission Path** 与 **Execution Path**，使参数接纳和副作用执行分别接受与其阶段相符的检查。

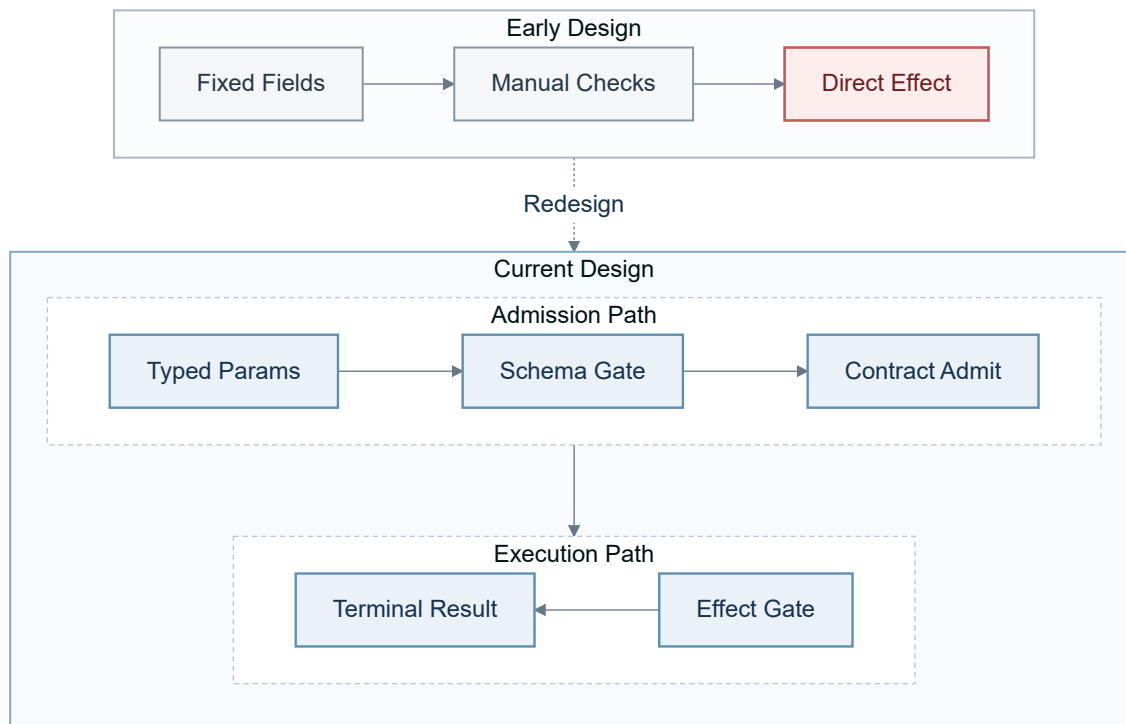


图 3.8: 工具调用从手写字段校验演进为执行约定与副作用门

图中上半部分的 Admission Path 依次处理 Typed Params、Schema Gate 和 Contract Admit: typed key-value 先通过 Registry Schema 校验, 再连同 node、attempt、摘要和资源声明写入本次 Contract。下半部分的 Execution Path 在副作用发生前经过 Effect Gate, 复核运行态声明后才产生 Terminal Result。参数校验负责判断输入结构, Contract Admit 固定本次尝试, Effect Gate 负责确认执行时刻仍满足原声明。接纳后已经取消、超时或变旧的请求会停在副作用门之前。

副作用门先核对 Contract 生命周期和代次, 再核对 node、request、attempt 与摘要。任意一项不再匹配时, 执行以 STALE 收敛。

代码 2: 副作用前的 Execution Contract 声明复核

c

```

1840 agent_execution_contract_effect_begin(struct agent_execution_claim *claim)
1841 {
1842     struct agent_execution_contract_record *record;
1843     struct agent_execution_node_runtime *runtime;
1844     int enabled;
1845
1846     if (claim == 0 || !claim->active)
1847         return AGENT_EXECUTION_EFFECT_STALE;
1848     if (claim->legacy)
1849         return AGENT_EXECUTION_EFFECT_ALLOWED;
1850     if (claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1851         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1852         return AGENT_EXECUTION_EFFECT_STALE;
1853     enabled = intr_save();

```

严格错误矩阵与 24 节点 Contract 回归覆盖 schema 漂移、非法前驱、越权、超时和 replay, 并检查资源结算最终回到零。

3.8.3. Context: 从 FIFO 截短到 active path 与 provenance

在初赛的设计中，Context 按 sequence 写入固定容量的 FIFO，并通过头指针和记录计数提供顺序查询。此时的 rollback() 直接退回最新序号、计数和头指针，使最近记录从可见历史中消失；记录字节当时仍留在环形区，后续追加会复用原有序号并逐步覆盖对应槽位。这种做法能够完成线性历史中的“撤销最近若干条记录”，却无法同时保留旧分支和新的决策路径。

随着工具重试、分支探索和跨轮对话进入同一个执行过程，Context 需要同时保存“曾经发生过什么”和“下一轮应继续使用哪条路径”。单纯延长 FIFO 只能扩大物理窗口，无法表示当前推理链选择的分支；直接截短又会丢失审计所需的旧记录。我们据此把分支代次、父记录序号和 prev_hash 纳入记录，将物理归档与 active path 分开，并增加 successor index，使完整路径查询能够沿选定分支完成一次正向扫描。

这次重构还需要解决并发读取与跨轮重建问题。Guest 镜像使用奇偶 publish sequence 区分写入中与已发布状态，读取方只有在前后观察到同一个偶数序号时才接纳记录。Nexus 将 USER、TOOL、FINAL 摘要绑定到 active path，下一轮消息直接从该路径重建；已经合并的 provenance 标签继续保持单调，切换可见路径不会降低既有来源标记。内核中的 Context 因而成为跨轮历史的统一来源。

FIFO 回退与 active path 分支的差异如图3.9所示。图中的 New Branch 表示可见路径从新的父记录继续延伸；Context rollback 切换可见头并分配新的分支代次，不会降低已经合并的 provenance 标签，也不撤销已经发生的文件写入、消息发送或外部服务副作用，需要补偿的外部行为仍由上层 workflow 负责。

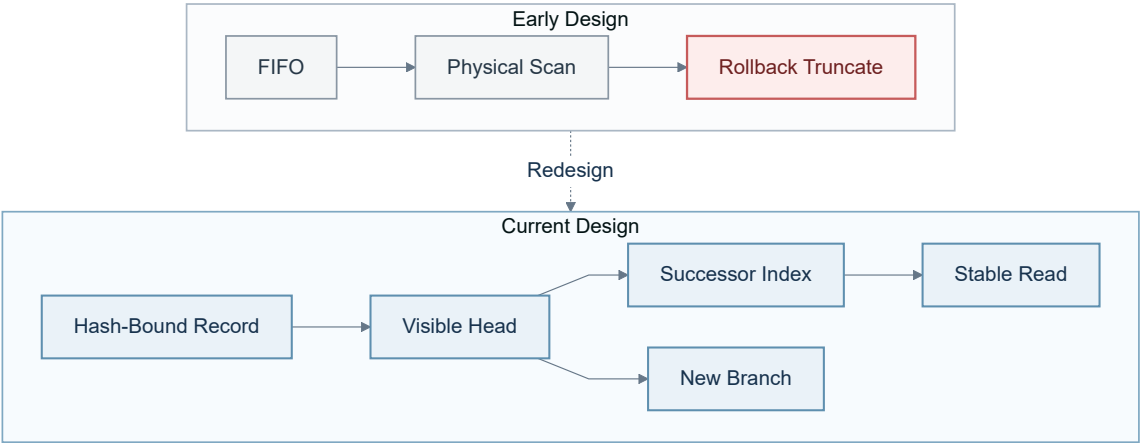


图 3.9: Context 从物理 FIFO 截短演进为可验证的 active path

图中上方的初赛路径把 FIFO、Physical Scan 和 Rollback Truncate 串在一起，回滚会让头指针、计数和 sequence 一起后退，原路径随后的记录失去可见入口，其槽位也可能在追加时被覆盖。下方当前路径先以 Hash-Bound Record 固定记录内容与前驱关系，再由 Visible Head 指出当前决策位置；读取沿 Successor Index 到达 Stable Read，新的决策则从同一可见头部进入 New Branch。旧分支因而可以留在归档中，当前路径又能独立向前推进。

路径读取同时检查记录是否处于有效窗口，并核对记录序号和 record_hash，避免被淘汰后复用的槽位或被破坏的记录进入当前路径。

代码 3: active path 记录的窗口、序号与哈希核验

c

```

91 static int
92 context_read_sequence(struct proc *p, uint64 sequence,
93                      struct agent_context_record *record)
94 {
95     uint64 slot;
96
97     if (p == 0 || record == 0 || p->context_path_capacity == 0 ||
98         sequence < p->context_path_oldest ||
99         sequence > p->context_path_latest)
100         return -1;
101     slot = (sequence - 1) % p->context_path_capacity;
102     if (agent_context_read_record(p, slot, record) < 0 ||
103         record->sequence != sequence || record->record_hash == 0 ||
104         record->record_hash != agent_context_record_hash(record))
105         return -1;
106     return 0;

```

回归覆盖回滚后归档保留、分支环路、哈希篡改、FIFO 淘汰和 provenance 单调性。验证结果表明旧记录在窗口内仍可查、新序号不复用，失败轮次也不会被带入下一轮模型消息。

在早期开发中，较长的 USER、TOOL 与 FINAL 正文集中保存在 Context 区第七页的 4 KiB 用户缓存中，内核路径只留下定长摘要。短任务能够据此衔接下一轮；工具输出增多以后，较早正文会被新投影覆盖，父任务也缺少可核验的完整子任务报告。当前实现增加用户态 Context Artifact Store：正文按 handle 保存，内核 syscall 571 核验类型、长度、UTF-8、producer 与 SHA-256，封存完成后才允许 Context 或 Task 终态引用。每个 Agent 保留 private Context，workflow 共享索引只引用已经结算并明确共享的 Artifact；达到高水位时，运行时提交保留 sequence、Task、revision、build 与 hash 的结构化摘要。

早期文件查询在收到模型请求后才读取目标，没有利用 active path 中已经形成的访问次序。当前实现让成功的只读查询同时提交机器可读签名，内核以固定 16 项转移表学习 $A \rightarrow B$ ，达到观察次数与置信度阈值后生成低优先级预取。训练只接纳 active path 上已经结算的记录；rollback、Context 清空、lifecycle、incarnation、revision 或 workspace generation 变化会清理相关状态。Guest 文件进入异步预取队列，Host 文件产生带 object id、revision 与 range 的提示，正式读取仍重新执行权限、root、revision 和 lifecycle 检查。

3.8.4. Catalog: 从逐项建档到批量复用与 Typed Watch

在初赛的设计中，我们让内核扫描普通目录，并把发现的文件属性写入 .agentmeta。这条路径随后加入递归扫描、查询缓存和 metadata 恢复，需要同时协调目录变化、持久记录、索引和前台查询。字符串 Watch 还要再次解释字段含义，初次查询与后续通知很难长期保持同一套条件。

在早期开发中，我们把自动扫描改为显式 Catalog，但综合场景仍在每次发现前逐项登记 96 条 Metadata，完成一次索引查询后立即清理。核心查询已经减少了候选记录，逐项系统调用、重复建档和一次性清理仍进入完整流程。性能日志显示核心窗口稳定缩短，完整流程没有保留这部分收益，问题由查询算法转向建档次数与 Catalog 生命周期。

我们据此分三步调整实现。第一步增加最多 16 项的批量 Metadata 接口，使一批对象共享一次生命周期操作和元数据事务，并按顺序返回逐项状态。第二步由 Guest 根据预计使用次数 K 选择路径： $K = 1$ 直接遍历， $K \geq 2$ 批量建档，并在同一 lifecycle 内复用 READY Catalog。第三步为文件订阅维护精确的进程存在计数，没有相关 Watch 时直接跳过进程表扫描。Typed Watch 继续复用 Live Query 的完整谓词，根据变化前后的集合成员关系生成事件。

文件发现与变化通知的演进如图3.10所示。图中将初赛自动扫描与字符串 Watch，同当前的 Discovery Path 和 Notification Path 放在同一流程中，说明查询对象从何而来、变化又如何传递给订阅者。

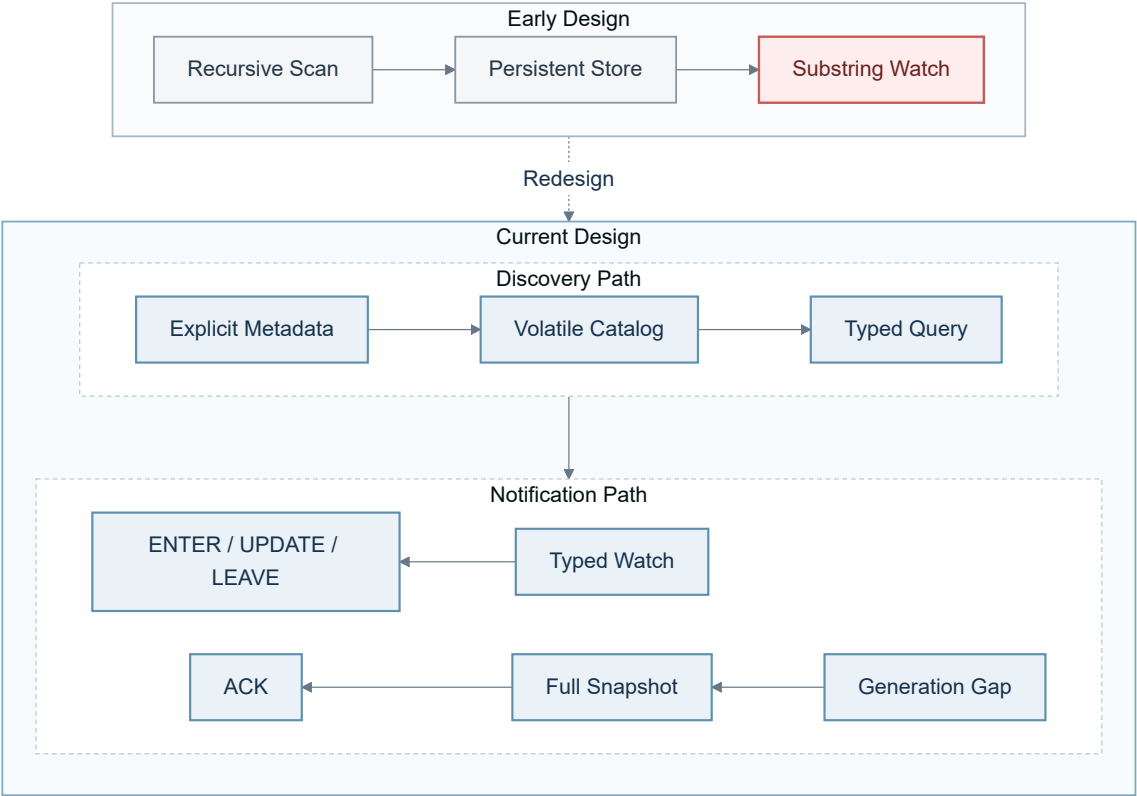


图 3.10: 文件发现从自动持久扫描收敛为显式 Catalog 与 Typed Watch

Discovery Path 从 Explicit Metadata 开始，经 Volatile Catalog 形成可查询集合，再由 Typed Query 按字段和类型筛选对象。Notification Path 使用同一个 typed predicate 驱动 Typed Watch，并根据变化前后的集合成员关系产生 ENTER、UPDATE 或 LEAVE；事件队列出现 Generation Gap 时，则按 Full Snapshot、ACK 的顺序完成重新同步。

当前事件类型由同一谓词在 before/after 两个状态上的真值组合直接确定，查询与通知共用同一份条件。下面的片段与这一集合语义一一对应。

代码 4: Typed Watch 的 ENTER、UPDATE 与 LEAVE 判定

c

```

596     before_matches = agent_live_query_watch_matches(
597         target, owner_scope, cold->watch_filter[slot], before);
598     after_matches = agent_live_query_watch_matches(
599         target, owner_scope, cold->watch_filter[slot], after);
600     if (!before_matches && after_matches)
601         change = AGENT_LIVE_QUERY_ENTER;
602     else if (before_matches && after_matches)
603         change = AGENT_LIVE_QUERY_UPDATE;
604     else if (before_matches)
605         change = AGENT_LIVE_QUERY_LEAVE;

```

从逐项登记、单次查询到批量建档与生命周期内复用的调整如图3.11所示。图上方的 Early Development 依次执行 96 Scalar Calls、One Query、Immediate Cleanup 和 Next Run Rebuild，同一组对象在下次查询前再次经历完整准备。图下方的 Current Design 先读取 Expected Uses: $K = 1$ 进入 Directory Walk; $K \geq 2$ 进入 Batch Register, 将 Catalog 推进到 READY, 后续查询从 Query Reuse 返回同一窗口。Lifecycle Reclaim 在工作流结束时集中回收; Corpus Change 进入 STALE, 随后回到建档路径。Zero-Watch Fast Path 位于批量登记与 READY 状态之间, 没有订阅时不进入进程扫描。

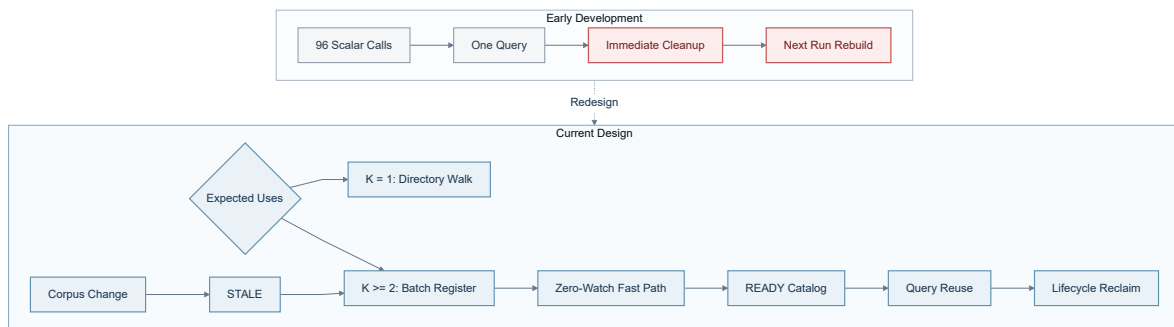


图 3.11: 文件查询从逐项单次建档演进为自适应批量建档与生命周期内复用

批量执行循环按输入顺序调用共享的 Metadata 变更核心, 先写回当前项状态, 再决定是否停止。普通业务错误不会撤销已经确认成功的前项; 无法确定提交结果时停止处理后续项目。

代码 5: 批量 Metadata 的逐项状态与失败前缀

c

```

1249     int stop_batch = 0;
1250     int item_status = agent_file_meta_set_execute(
1251         p, 0, &agent_file_meta_batch_scratch[i], scope_id, 1,
1252         &stop_batch);
1253
1254     if (item_status == -1)
1255         break;
1256     if (copyout(p->pagetable,
1257         statusesaddr + (uint64)i * sizeof(item_status),
1258         (char *)&item_status, sizeof(item_status)) < 0) {
1259         processed = AGENT_STATUS_INDETERMINATE;
1260         break;
1261     }
1262     processed++;
1263     if (stop_batch || item_status == AGENT_STATUS_INDETERMINATE)
1264         break;
1265     agent_metadata_txn_work_charge(1);
1266 }

```

事件发布路径先完成对象和 lifecycle 核验，再读取受中断保护的 Watch 存在计数。计数为零时，下面五行直接结束通知处理，不再遍历全部进程。

代码 6: 没有文件订阅时跳过进程扫描

c

```

1323     if (!agent_live_query_domain_valid(key, owner_scope) ||
1324         identity == 0 || !identity->used || identity->fid <= 0)
1325         return 0;
1326     if (!agent_live_query_file_watches_present())
1327         return 0;

```

回归确认普通文件不会自动入表，并覆盖批量混合状态、失败前缀、三类事件、缺口恢复、zero-watch 快速路径与 inode incarnation。在 $K = 4$ 的 16 次独立 QEMU 启动中，96 条记录通过 6 次批量调用登记，Catalog 构建一次并完成 4 次复用。核心窗口与完整流程均为 16/16 个配对更快，配对中位差分别为 -105.668 ms 与 -89.782 ms ，逐项登记和一次性清理带来的完整流程问题得到解决。

3.8.5. 任务委派：从 N1 MESSAGE 到 native Task Channel

在初赛的设计中，Nexus 已能通过通用的 N1: MESSAGE 把 child Task 交给另一个 Agent。消息携带任务编号和操作类型，发起者在用户态记录发送、等待与返回状态。这种做法可以串起基本委派流程，但 MESSAGE 只表达一次通信，任务是否被领取、执行者属于哪一代以及最终由谁完成仍由多处状态共同判断。

加入取消与硬截止时间后，发起者还要发送 CANCEL 消息、唤醒目标并过滤迟到结果。完成消息与取消消息可能以不同顺序到达，用户态需要重复判断哪个结果有效；N1 已经携带生命周期代次和 deadline，大段正文也保存在 capsule artifact 中，但通用 MESSAGE route 仍不记录 claim 的线性化时点，也无法在内核中裁决完成、取消与超时中的唯一终态。

针对这些问题，我们让 Nexus 接入并扩展已有的 native Task Channel，以 pending、claim 和 complete 三个阶段组织委派。128 字节 descriptor 绑定 target、parent task、目标与输入

Artifact、所需 capability、允许工具、workspace revision、资源预算、deadline 和结果类型；owner 以及通道、请求和槽位代次由内核槽位与 claim/complete 回执绑定。大段正文继续保留在已封存 Artifact 中，控制描述符只负责传递不可变关联。内核检查父子身份、授权包含关系与任务图，允许一个父任务并行创建多个无环子任务。取消、超时与正常完成最终进入同一个内核终态交付过程；任务收敛到 READY 后，owner 再通过 enter 接口取得已经结算的 terminal CQE。

Nexus 委派从用户态状态过滤到内核终态结算的演进如图3.12所示。当前方案将 native Task Channel 分为 Submit Path 与 Finish Path，使任务领取和终态交付围绕同一个内核任务槽位推进。

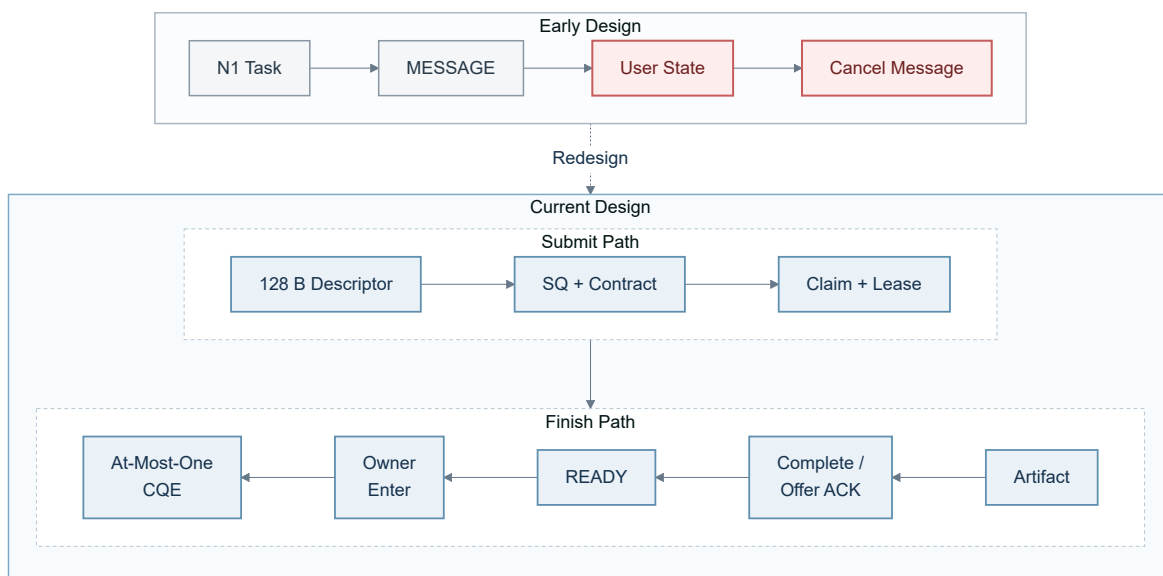


图 3.12: Nexus 委派从 N1 MESSAGE 演进为原生 Task Channel

Submit Path 由 128 B Descriptor 进入 SQ 与 Contract，再通过 Claim 与 Lease 把任务绑定到领取者。Finish Path 从右向左推进：worker 先封存正文结果 Artifact，正常完成时提交 Complete；若内核已经给出 terminal offer，则清理任务并回送 Offer ACK。两种情况都先收敛到 READY，随后由图中的 Owner Enter 表示 owner 调用 enter 接口，至多取得一条 terminal CQE。取消沿同一状态机进入终态，和 provider 完成路径共用一次交付机会，不会额外生成第二条完成记录。

worker 完成任务时，内核会复核领取回执中的生命周期、进程、线程代次以及完整任务 tuple。下面 20 行比较正是“领取者”和“完成者”不能被拆开的核心约束。

代码 7: delegated Task 完成回执的完整绑定核验

c

```

489 agent_task_delegate_receipt_binding_matches(
490     const struct agent_task_delegate_receipt *receipt,
491     const struct proc *worker, const struct thread *thread,
492     const struct agent_task_delegate_complete *complete,
493     struct workflow_lifecycle_key lifecycle)
494 {
495     return receipt != 0 && receipt->valid && worker != 0 && thread != 0 &&
496         complete != 0 &&
497         workflow_lifecycle_key_equal(receipt->lifecycle, lifecycle) &&
498         receipt->worker_pid == worker->pid &&
499         receipt->worker_agent_id == worker->agent_id &&
500         receipt->worker_control_id == worker->agent_control_id &&
501         receipt->worker_thread_generation == thread->identity_generation &&
502         receipt->owner_pid == complete->owner_pid &&
503         receipt->owner_control_id == complete->owner_control_id &&
504         receipt->channel_generation == complete->channel_generation &&
505         receipt->request_id == complete->request_id &&
506         receipt->slot_generation == complete->slot_generation &&
507         receipt->task_id == complete->task_id &&
508         receipt->correlation_id == complete->correlation_id;

```

Task Channel 与 Harness 回归覆盖 claim 权限、取消和 deadline 竞争、terminal offer/ACK 及资源回收，并验证任务进入 READY 后，owner 通过 enter 接口至多取得一条 terminal CQE。该唯一性由单个 Guest 内核中的通道状态机保证；当前委派协议仍依赖已领取任务的 provider 协作确认终态，无法自动终结永久无响应的执行者。

3.8.6. workflow 服务：从线程份额到 Credit Domain 与 EEVDF

在初赛的设计中，我们沿用 uCore 的线程级取队方式，并把资源额度分别记在进程或具体对象上。对于单 workflow 或线程数接近的场景，这种做法能够直接复用原有调度与回收流程；多个 Agent workflow 同时运行后，同一 workflow 增加线程就可能获得更多入队机会，使线程数意外成为 CPU 服务权重。

资源记账也出现了相似的聚合问题。创建中的额度、使用中的额度和退出中的延迟释放若由各对象分别结算，workflow 结束时就难以一次回答还有多少额度被占用、还有多少操作尚未离开。性能与生命周期分析由此得到两个需要同时解决的问题：CPU 公平性应以 workflow 为计量实体，资源回收也应在 workflow 层形成可等待的完成条件。

当前实现先把八类内核对象的资源额度收敛到 workflow 的 free/pending/used 账户。Workflow Fence 在活动操作和离开中的操作归零时尝试取得静止点：它暂停新操作，推进当前文件系统延迟回收，收缩空闲额度并要求 pending credit 归零，再形成阶段快照；条件尚未满足时立即返回 RETRY。CPU 服务则由 workflow EEVDF 单独结算：系统以 workflow 为实体累计 service、vruntime 与 virtual deadline，选中 workflow 后再从其内部挑选可运行线程。这样，外层调度只比较 workflow 实体，内部线程数量不会直接生成额外权重，资源核算与 CPU 服务也能共同使用 Lifecycle Key 定位对象。

资源结算与 CPU 服务收敛到 workflow 粒度的结构如图 3.13 所示。当前方案以 Lifecycle Key 为共同索引，将额度、退出等待和 CPU 服务拆成三条可独立推进又指向同一 workflow 的路径。

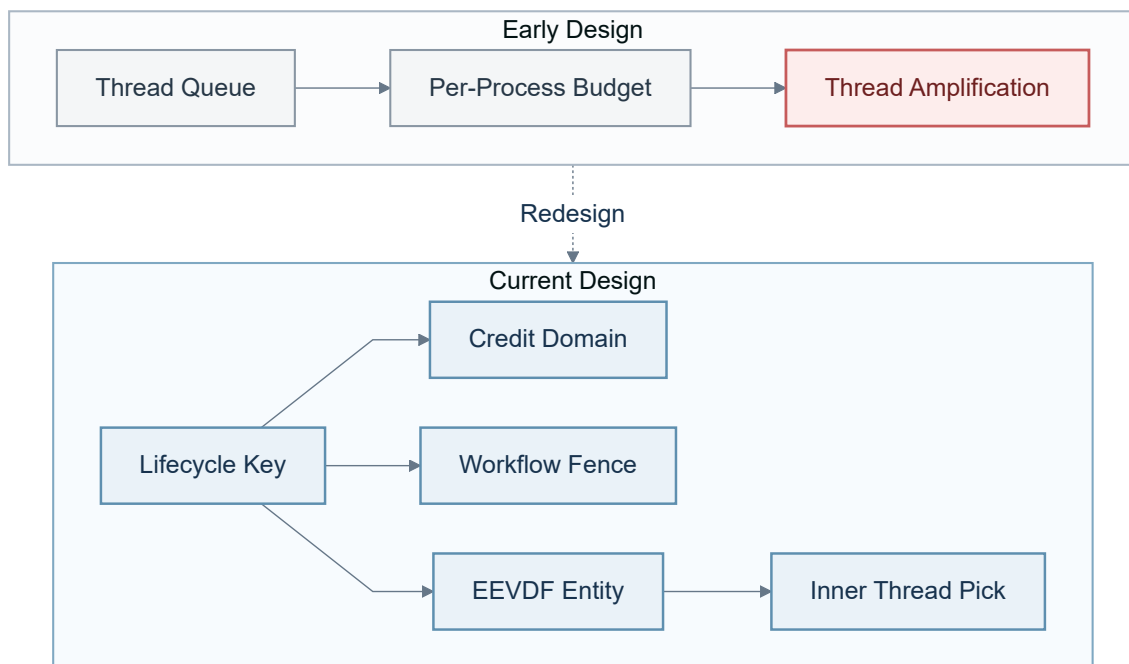


图 3.13: 资源额度与 CPU 服务从线程粒度收敛到工作流粒度

图中上方的初赛路径从 Thread Queue 进入 Per-Process Budget，线程数量会间接放大可获得的服务。下方当前路径从 Lifecycle Key 分出三路：Credit Domain 维护 free/pending/used 额度；Workflow Fence 尝试在活动操作和 pending credit 归零时取得阶段快照，暂时无法取得便返回 RETRY；EEVDF Entity 先选择工作流，再进入 Inner Thread Pick。前两路回答资源是否可以继续使用以及当前能否完成阶段结算，第三路决定下一段 CPU 服务交给哪个工作流。

下面的记账片段给每个工作流实体使用固定权重，并以实际服务量推进 vruntime 和 virtual deadline。新增线程不会在这一层生成新的独立权重。

代码 8: workflow EEVDF 的服务量与虚拟截止时间记账

c

```

1155     entity->service_cycles = workflow_scheduler_counter_add(
1156         entity->service_cycles, service_cycles);
1157     /* All workflow entities have one fixed weight, independent of threads. */
1158     entity->vruntime = workflow_scheduler_add_sat(
1159         entity->vruntime, service_cycles);
1160     if (service_cycles >= entity->remaining_cycles)
1161         entity->remaining_cycles = 0;
1162     else
1163         entity->remaining_cycles -= service_cycles;
1164     entity->virtual_deadline = workflow_scheduler_add_sat(
1165         entity->vruntime, entity->remaining_cycles);
  
```

六次 QEMU 启动中的 504 次唤醒均落在 0 至 1 tick，1 至 4 个并发工作流的 Jain 指数中位数均不低于 0.999985。在四个工作流竞争、其中一个使用四线程的场景中，该工作流份额仍为 26.895%，高于四等分的 25%，说明当前实现基本抑制了多线程放大，但仍存在小幅偏差。

3.8.7. Nexus 路径演进与通用 Harness 接管

在初赛的设计中，Host 在启动前构建并固定受限源码 corpus，Guest Research 通过 N1 child Task 在这份快照中执行 `source_search/source_read`，Host 再按同一 corpus 重放检索并核对 citation。该方案已经把搜索动作放进 Guest，却仍需为一次演示预先打包受限源码 corpus；工作区发生变化后，需要重新生成 corpus、装入 Guest 镜像，并按新的 revision 生成引用。

在早期开发中，我们曾把工作区访问改成另一条过渡路径：Guest Research 先产生搜索或读取请求，Host WorkspaceReader 执行操作，再把工具结果直接写入 Provider history。这解决了完整快照的更新成本，却让候选选择和跨轮正文重新集中到 Host；Guest 内的 Catalog、Task Channel 与 active path 无法独立复现“选择了哪个文件、哪个任务完成、下一轮使用了什么结果”。

经过职责拆分，固定角色 Nexus 曾将 Host 收敛为 broker：Host 分页提供 versioned manifest，Guest 用 Metadata Catalog 与 Live Query 选择候选，native Task Channel 负责委派。该路径证明了 Host 数据可以作为受版本约束的输入进入 Guest，但角色、串口状态机与产品命令仍然绑定在一个大型应用中。

当前实现进一步下线固定角色与早期 Nexus 入口。通用 Harness 为整个会话启动一个长期运行的 `agentharness_ucore` Guest；每个 Host Agent 都由 Guest 的 `agent_runtime_control` (SPAWN) 建立对应进程，root Task、动态子 Task、claim、complete 与 terminal CQE 全部进入该 Guest 的原生 Task Channel。Host Artifact Store 保存完整正文，Guest 内核维护身份、生命周期、任务状态和终态交付。工作区批量 Catalog、Typed Watch 与稳定窗口机制也进入通用 `search_files/read_file` 产品路径：Host 提供版本化 manifest，Guest 先形成受内核核验的候选，再允许 Host 读取候选正文。

固定角色 Nexus 退役前的职责拆分如图3.14所示。该图记录 versioned manifest、Catalog、Typed Watch 与 Task Channel 如何完成一次联合验证；当前通用 Harness 复用其中的 broker、Artifact 与 Task 约束，并用长期 Guest 取代固定角色状态机。

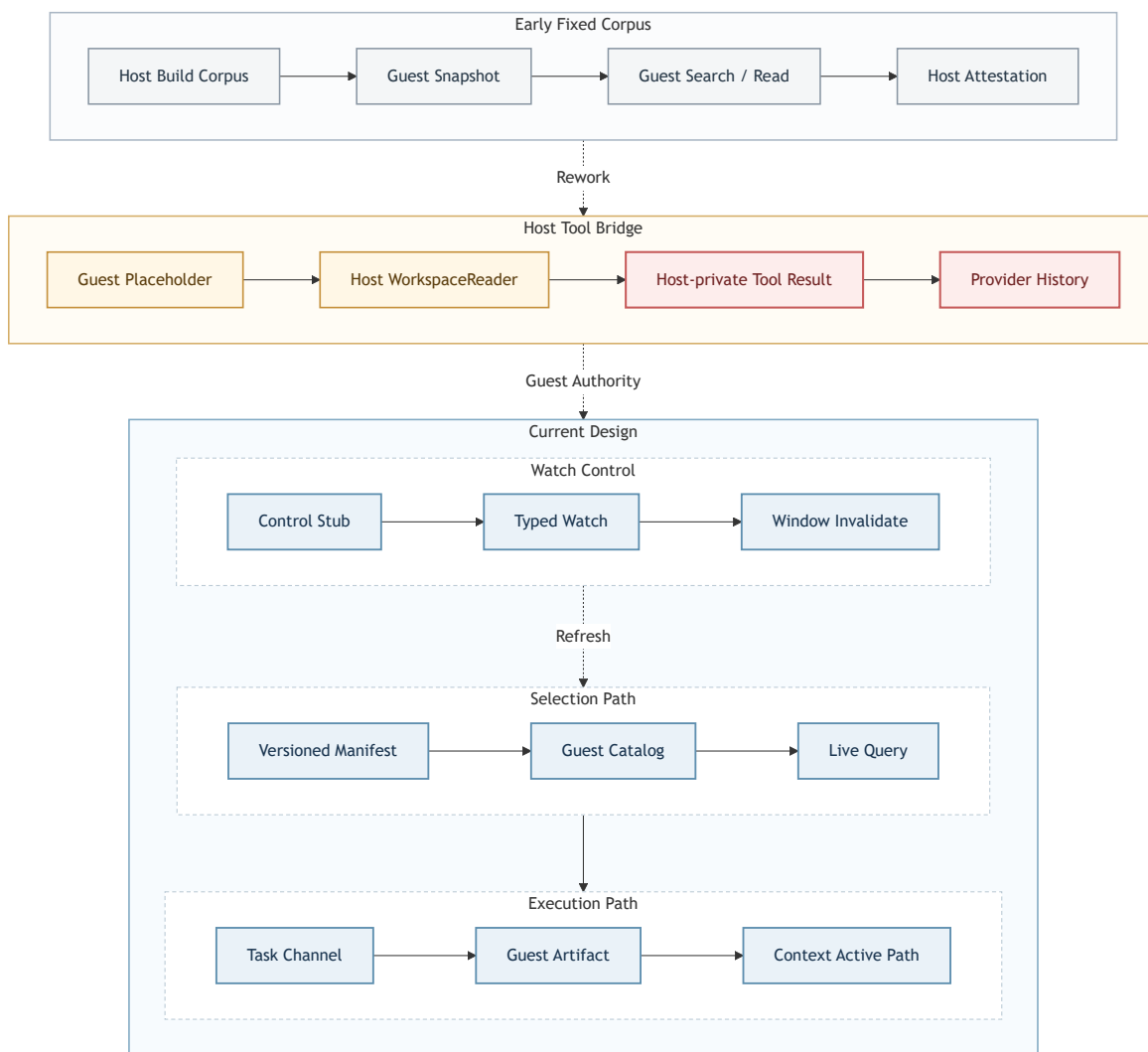


图 3.14: 固定角色 Nexus 退役前的 Host Broker 与 Guest Authority 演进

图上方依次呈现两条早期路径。固定 corpus 路径先由 Host Build Corpus 生成 Guest 快照，随后 Guest 完成 Search / Read，Host 再核对 Attestation；过渡路径由 Guest Placeholder 发起请求，经 Host WorkspaceReader 得到 Host-private Tool Result，并直接进入 Provider History。图中的收敛路径让 Versioned Manifest、Catalog、Live Query、Typed Watch、Task Channel、Guest Artifact 和 Context Active Path 形成可查询链路。当前通用 Harness 使用这条收敛路径完成搜索与读取，并把所有 Agent 对应进程和工具 Task 放入一个长期运行 Guest。

下面的代码展示通用 Harness 如何在 Guest 中创建与 Host 配置对应的受控 Agent。用户给出的 capability 与工具集合先经过 policy 校验，Guest 再补充 provider 进程完成 Task transport 所需的最小内部能力。

代码 9：长期 Guest 中的通用 Agent 创建

c

```

613     request.flags = AGENT_TASK_CHANNEL_SETUP_F_SINGLE_ISSUER;
614     request.lifecycle = lifecycle;
615     if (agent_task_channel_setup(&request, &result) != 0 ||
616         result.status != AGENT_TASK_CHANNEL_OK || result.generation == 0)
617         fail("task_channel_setup");
618     channel.generation = result.generation;
619     channel.sq = (volatile struct harness_sq_page *) (unsigned long) result.sq_base;
620     channel.cq = (volatile const struct harness_cq_page *) (unsigned long) result.cq_base;
621 }
622
623 static void prime_context(void)
624 {
625     struct agent_op operation;
626     struct agent_result result;
627
628     memset(&operation, 0, sizeof(operation));
629     memset(&result, 0, sizeof(result));

```

Host broker 负责连接在线 Provider，并读取受路径、版本和长度约束的工作区字节；Guest 内核保存身份、Task 状态和终态。下面的代码把动态 descriptor 编码为 Guest 命令并等待内核 claim 回执，Host 只在 identity 与 task id 全部匹配后继续模型循环。

代码 10：Host 动态 Task 到原生 Guest 的委派

Python

```

225         self._stderr.append(kept)
226         self._log_bytes += len(kept)
227
228         threading.Thread(
229             target=run,
230             name="agent-harness-native-stdout" if stdout else "agent-harness-native-stderr",
231             daemon=True,
232         ).start()
233
234     def _write(self, line: str) -> None:
235         if self._closed:
236             raise NativeTaskChannelError("native_guest_closed")
237         relay._write_process_before_deadline(
238             self.process,
239             line.encode("ascii") + b"\n",
240             deadline_monotonic=time.monotonic() + 5.0,
241         )
242
243     def _wait_prefix(self, prefix: str, deadline: float) -> str:
244         while True:

```

4. 项目测试

测试先核对接口结构和构建结果，再进入 RISC-V Guest 检查各项内核机制，最后运行控制台、Nexus 和相应的性能负载。功能结果来自真实系统调用输出和协议摘要；性能数据保留逐样本来源，配对比较、分布图和统计表都能追溯到对应的启动记录。

4.1. 功能与性能测试

4.1.1. 测试组织

AgentOS-uCore 的测试从接口结构、内核镜像、Guest 行为和综合场景四个方向交叉检查。结构约定检查核对 UAPI 的尺寸、偏移和模块依赖；ABI vNext 的冻结布局清单覆盖 702 项 UAPI 合约，其中包含 33 项 Tool Registry、128 字节 delegated Task 描述符、Context Artifact、查询预测与系统调用 567–572，并确认恒零字段、V1 请求和退役包装已经移除。交叉编译把真实内核和用户程序链接为 RISC-V 镜像；Guest 场景通过系统调用观察 Agent 身份、Context、VFS、IPC、资源和调度状态；Host 测试覆盖 Provider、工作区 broker、Artifact、通用 Agent Loop、原生 Task Channel adapter 与验证脚本。性能测量直接读取同一 Guest 工作负载产生的时钟和计数。

对应机制	主要程序	观察重点
Agent 进程与 Context 区	agenteval_ucore, agenttrust_ucore, agentscope_ucore, agentfinal_ucore	创建、派生、装入和退出；固定映射、直接读取、回滚、清空与 FIFO 淘汰
结构化接口与工具协议	agenteval_ucore, agenttoolabi_ucore, agentcontract_ucore, agenttask_ucore	33 项目录与 schema；废弃、syscall-only 与 brokered 状态；V2/V3；128 字节动态 Task；claim/complete、终态确认、至多一条 CQE、背压与重新同步
文件查询与实时事件	agenteval_ucore, agentfs_ucore, agentbench_ucore	元数据与摘要；遍历和索引结果一致性；ENTER、UPDATE、LEAVE 与缺口恢复
Agent Loop 与 EEVDF	agenteval_ucore, agentloop_ucore, agentsched_ucore, agent_eevdf_ucore	原子等待、消息与 TIMER 唤醒、超时、线程放大、工作流服务份额
通用 Harness 与综合场景	labdemo_ucore, agentharness_ucore, agentmulti_ucore	长期 Guest、动态 runtime config、原生 Task Channel、Artifact seal/share、查询预测、版本化读写、隔离构建、独立 Guest 用例与关闭

表 4.1: 系统机制、测试程序与观察重点

自动化检查依次覆盖智能体模块与 ABI 结构约定、通用 Harness 合约和 Host 测试程序。agentos-harness-native-test 在一个长期运行 Guest 中创建与 Host 配置对应的 Agent，提交 root Task 与嵌套子 Task，并检查 claim、complete、terminal CQE 和正常关闭。开发 replay 另行重放写入、失败编译、修补、成功构建和三类 Guest 结果，检查旧 build 失效与最终完成条件。agentmulti_ucore 在真实 Guest 中验证 runtime 权限子集、Artifact 封存/绑定/共享、 $A \rightarrow B$ 预测、Host hint 与命中记账；Host 测试验证共同 Agent Loop、动态子任务图、能力包含关系和结构化摘要。

```
1 make agent-module-check
2 make agentos-build TOOLPREFIX=riscv64-linux-gnu-
3 make agentos-nexus-check TOOLPREFIX=riscv64-linux-gnu-
4 make agentos-harness-native-test TOOLPREFIX=riscv64-linux-gnu-
5 make local-host-selftests
```

DeepSeek 计算器任务用于验证 Harness 的受控开发 broker 能够持续执行“阅读、创建、编译、测试、总结”。模型自行选择单 Agent 方案，在 5 个模型轮次中调用 4 次产品工具；源码 revision 前缀为 c3f019cfc7e7，最终 build id 前缀为 17a5ea4b2fc8。同一 build 的 5 项运行结果见表 4.2；每一行都来自独立的 AgentOS-uCore Guest。读取、写入、构建和运行分别绑定长期控制 Guest 中的 native Task，工具结果经 Artifact 与 Context 交付。

用例类型	串口输入	实际程序输出	退出状态
正常表达式	2+3*8/6+5*2	16	0
非法字符	2+a*3	error: invalid expression	1
空输入	空行	error: empty input	1
运算符错误	2+*3	error: invalid expression	1
关键失败路径	5/0+1	error: division by zero	1

表 4.2: DeepSeek 计算器开发任务的独立 Guest 运行证据

5 项运行均记录 output_match=1、退出状态匹配并且未超时，session 最终正常关闭。会话封存 10 个 Artifact，Workflow Fence 汇总 181 项 evidence event；Catalog 状态记录 6 项、候选 5 项和 58 次 Typed Watch 事件。开发完成门在运行证据绑定当前 source revision 和 build id 之前拒绝 FINAL，任何后续 write_file 或 apply_patch 都会清空旧 build 及用例集合。在线证据索引保存在 ci/agentos-nexus-multiagent-evidence.json，确定性失败编译、修补和重测流程另由 ci/agentos-nexus-dev-replay.jsonl 覆盖。

4.1.2. 任务一至任务五的 Guest 综合验收

agenteval_ucore 把赛题任务一至任务五放在同一个 RV64 Guest 中检查。运行命令如下：

```
1 AGENT_TEST_CASE=agenteval_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

这项测试以系统调用后的 Guest 可见状态为判据，五段场景的观察内容见表4.3。

这组综合验收把创建、工具调用、Context、文件查询和 Loop 串在同一次启动中，能够发现单项测试不易暴露的状态衔接问题。例如，Context 映射若只在创建时存在、exec 后丢失，后续的查询缓存场景便无法继续；文件事件若没有正确唤醒等待者，Loop 场景也不会仅凭查询结果“看起来正常”。

4.1.3. 结构化工具与任务通道测量

任务通道测量让紧凑 Batch、单项 V3 和 SQ/CQ 分别执行 16 次等价 ECHO，并轮换三条路径的先后顺序。每条操作序列记录完整墙钟时间，同时核对工具、状态、Context 序号、执行记录票号和结果指纹，确保三条路径完成等价工作。4 次启动共保留 96 条操作序列。

场景	Guest 中可观察的行为
Agent 进程创建与地址空间	Agent 与普通进程可以并存；身份和配额可查询；固定高地址的七页 Context 区中，六页内核镜像保持只读，用户缓存页可写。
内核结构化交互接口	Guest 能列举工具并发起多种带类型信息的调用；未知工具、错误参数类型和小返回缓冲区得到不同的结构化状态。
上下文路径管理	六轮以上连续调用的直接映射读取与系统调用查询一致；回滚和清空改变后续可见路径；连续写入 133 条记录后只保留最新 128 条，系统继续运行且未耗尽内存。
面向 Agent 的文件查询	文件按属性和摘要被找到，不要求调用者预先知道完整路径；遍历与索引返回同一结果；结构化结果可以写入第七页用户缓存。
Agent Loop 内核运行机制	等待线程在无事件时睡眠，消息或 TIMER 到达后继续运行；多个 Agent 同时活动时， workflow 调度统计持续推进。

表 4.3: agenteval_ucore 的五段可观察行为

三种任务提交方式的主体分布与尾部耗时如图4.1所示。图 a 以散点、箱线和半侧密度展示 Batch、单项 V3 与 SQ/CQ 各 32 轮“16 次等价操作”的完整耗时，标注的 P50 用于比较主要样本集中区域；图 b 在对数横轴上给出三条路径的累积分布，并以同色圆点标出 P95，使首轮高耗时样本形成的长尾不会被主体分布遮住。两幅图共同呈现常态开销和高分位响应。

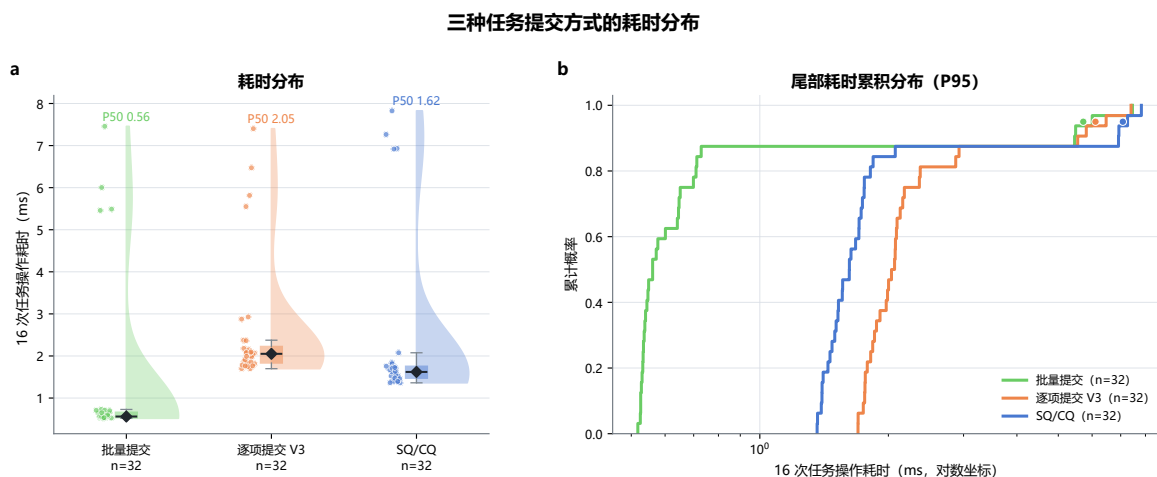


图 4.1: 三种任务提交方式的耗时分布

图 a 中，紧凑 Batch、单项 V3 和 SQ/CQ 完成 16 项等价操作的中位数分别为 561 微秒、2051 微秒和 1620.5 微秒。三条路径分别使用 1、16、2 次系统调用：Batch 在一次内核进入中顺序执行短操作；V3 为每一项绑定执行约定、节点和尝试号；SQ/CQ 通过共享队列批量提交并回收结果。系统调用次数的排序与主体分布的耗时排序一致。

图 b 中三条曲线在主体样本之后继续延伸到毫秒级尾部。Batch、单项 V3 和 SQ/CQ 的首轮中位数分别为 5.747、6.1435、7.098 ms，后续轮次降至 0.548、1.9965、1.5615 ms；各路径在累计概率约 0.875 处进入稀疏尾部，也与每条路径保留 4 个首轮样本、28 个后续样本的采集安排相呼应。刚进入任务路径时的结构建立和缓存状态会明显抬高延迟，因此评估同时保留首轮与后续轮次，部署时则应复用已经建立的通道处理连续任务。

这一结果使三条路径形成明确分工：短小、同质且需要同步返回的操作使用 Batch；需要逐项绑定 Execution Contract、节点、尝试号和执行记录时使用 V3；需要持续队列、

背压、取消和唯一 terminal CQE 时使用 SQ/CQ。该批数据覆盖本地等价 ECHO，跨 Agent claim/complete、长 Provider 延迟与异步 backlog 另行压测。

4.1.4. 结果文件发布与冲突处理

结果文件发布测试直接在 Guest 中调用 `agent_file_publish()`，同时检查正常写入、同名竞争、错误参数和 Nexus 收敛逻辑。运行命令如下：

```
1 AGENT_TEST_CASE=agentpublish_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

正常路径读取到完整 header、payload 和 EOF。同一访问范围内的两个调用并发发布同名文件时，结果恰好为一个 OK 和一个 DUPLICATE，已经发布的内容没有被覆盖。坏指针、非法路径、错误结构大小、错误版本和非零保留字段均未留下正式目录项；失败与重复请求也没有增加 workflow 资源计数，删除测试生成的两份文件后，块和 inode 计数回到基线。

Nexus 随后按正式路径逐字节回读。已有内容与本次 header、payload 和 EOF 完全一致时，发布结果可以收敛为成功；内容不同则明确拒绝。这组结果表明，未命名 inode 与单次正式目录接入避免了“文件名已经可见、内容尚未写完”的中间状态，同名竞争也没有演变为静默覆盖。失败和重复发布同样沿用 workflow 资源账户的记账与回收路径。

通用 FS epoch 动态回归在 dirty、inflight 和 durable 三种状态下均通过，覆盖有序批提交、fsync、重启重放和既有 power-cut 窗口。作为共用的分配、回收和恢复基础，分配器故障与重启矩阵的 36 个用例也全部得到预期状态；去掉关键 FLUSH 的变异会被测试捕获。结果文件发布沿用这条 checkpoint 与恢复路径。

4.1.5. 文件查询路径的 I/O 观测

查询测量同时记录文件读写、系统调用、扫描记录数、任务完成次数和 workflow 服务次数。如图 4.2 所示，图 a 比较遍历查询与索引复用：bytes_read 取自核心查询窗口，目录、块设备和 VirtIO 计数取自完整流程，再分别按核心窗口内的系统调用次数归一化；图 b 按已完成操作数比较 Batch、单项 V3 和 SQ/CQ，指标依次覆盖系统调用、ABI 描述符及复制、派发头、控制 ABI 及复制、调度次数和序列耗时。

每个单元格给出归一化后的中位数与样本量，颜色按同一指标列内的最大值缩放。因此，横向阅读用于辨认一条路径的开销构成，纵向阅读用于比较不同路径在同一指标上的相对大小。



图 4.2: 各查询与任务路径的内核 I/O 观测

图 a 显示选择性索引先缩小候选集，再完成相同条件复核，从而减少核心查询中的无关读取；图 b 显示 Batch 以更少的内核往返完成同质短操作，V3 与 SQ/CQ 分别保留逐项执行语义和持续队列语义。正式 campaign 将 96 条元数据记录收敛为 6 次批量调用，并在一次 Catalog 冷建后复用 4 次查询，检查记录数由 385 降至 5。结合图 3.5 的完整流程配对结果，这些观测说明批量登记和复用同时缩短了查询准备与后续发现阶段。

图 b 进一步解释了三种任务路径的传输成本。Batch 每 16 项只进入内核一次，平均为 0.0625 次/操作，每项携带并复制 224 字节 ABI 描述符；V3 逐项进入内核，描述符与复制量均为 768 字节，并增加 8 字节派发头；SQ/CQ 平均为 0.125 次系统调用和 256 字节描述符复制，并为每项保留 21 字节控制 ABI 与 34 字节控制复制。Batch 以最少往返服务同步短操作，V3 保留完整的逐项语义，SQ/CQ 则用少量控制数据换取持续队列和异步管理能力。

Batch、单项 V3、SQ/CQ 分别有 7/32、9/32、6/32 轮跨过 1 个 tick。10 ms tick 用于观察调度节拍，微秒级差异以墙钟测量为准。

4.1.6. 测量数据与可重复性

正式性能 campaign 使用 16 次相互独立的全新赛题平台启动，设置 $K = 4$ 次发现查询，A/B 与 B/A 顺序各 8 组。每条记录都带有原始数据文件名、文件 SHA-256 和原文序号；同一次启动中的内部配对用于观察相同工作量下的波动，跨启动汇总时以源启动记录作为独立单位。遍历路径与索引复用路径的结果哈希完全一致。

数据清单记录测试环境、工作量计划和数据文件校验信息，并汇总原始输出、CSV 数据表和图像文件哈希；清单项均通过哈希校验。图表中的每个点都可以回溯到对应的 Guest 输出。

对应机制	测量设计	独立单位	样本量
综合恢复与文件查询	16 个 A/B、B/A 各 8 组的 $K = 4$ 均衡配对启动	启动记录	16 组核心路径和完整流程配对
元数据目录与索引	3×4 参数网格，每格 15 个内部配对	源启动记录	180 组查询配对
工具协议与任务通道	4 次启动，8 组轮换顺序	启动记录/区段	96 条 16 次操作序列
Agent Loop 与 EEVDF	6 次启动，并发度为 1-4	启动记录/时间窗和探针	504 条精确唤醒探针

表 4.4: 各内核机制的测量数据构成

4.2. 测试路径与结果

测试通过真实 RV64 Guest 调用观察内核对象的状态、返回码、上下文序号、代次和运行统计。功能场景、机制探针和性能测量共用同一构建产物，表4.1中的项目均对应实际执行路径。

4.2.1. Guest 原生工作负载

labdemo_ucore 使用同一组语料完成“定位失败阶段、写回恢复结果、再次确认状态”的业务流程。下列遍历内层循环会为每个候选读取正文、累计检查记录数，再确认唯一失败阶段。

代码 11: 遍历路径逐文件扫描

c

```
658     for (int i = 0; i < LABDEMO_CORPUS_SIZE; i++) {
659         int fd;
660         ssize_t bytes;
661
662         demo_corpus_name(name, 'c', i);
663         fd = open(name, O_RDONLY);
664         check(fd >= 0, "compat corpus open");
665         memset(buffer, 0, sizeof(buffer));
666         bytes = read(fd, buffer, sizeof(buffer) - 1);
667         check(bytes > 0, "compat corpus read");
668         metrics->bytes_read += (uint64)bytes;
669         metrics->records_examined++;
670         check(close(fd) == 0, "compat corpus close");
671         if (strcmp(buffer, DEMO_BENCH_FAILED_BODY) == 0)
672             found++;
673     }
674     check(found == 1, "compat unique failed stage");
```

索引复用路径先把 96 条元数据记录分成 6 次批量调用。代码中的短循环等待本组已处理条目，并逐项检查返回状态；全部条目成功后，Catalog 进入可查询状态。

代码 12: Catalog 的批量登记与状态核验

c

```

805     while (completed < count) {
806         int processed = agent_file_meta_set_batch(
807             &native_meta_batch[completed],
808             &native_meta_status[completed], count - completed, 0);
809
810         session->batch_calls++;
811         if (processed <= 0 || processed > count - completed) {
812             session->state = LABDEMO_CATALOG_DIRTY;
813             session->cold_build_us =
814                 demo_now_us() - started_us;
815             return -1;
816         }
817         for (int i = 0; i < processed; i++) {
818             if (native_meta_status[completed + i] !=
819                 AGENT_STATUS_OK) {
820                 session->state = LABDEMO_CATALOG_DIRTY;
821                 session->cold_build_us =
822                     demo_now_us() - started_us;
823                 return -1;
824             }
825         }
826         completed += processed;
827         session->registered_items += processed;
828     }

```

Catalog 冷建 1 次后在后续 4 次查询中复用，同一工作流的检查记录数由 385 降至 5；遍历与索引复用路径仍执行相同的恢复写入，并生成相同的结果哈希。如图 3.5 所示，图 a 对比核心窗口，图 b 对比完整流程，图 c 汇总两者的配对差值；三个面板直接读取当前测量目录的时间与资源数据。

4.2.2. 工具、执行约定与任务通道测试

通用 Harness 发起跨 Agent 工具调用时，把目标、输入 Artifact、capability、允许工具、workspace revision、资源预算、deadline 和结果类型写入 128 字节不可变描述符。描述符导入 Task Channel 后，由 delegate_task SQE 绑定 Execution Contract 并提交。

代码 13: 长期 Guest 中的 Task 描述符与 SQE 构造

c

```

713     *effects |= AGENT_SIDE_EFFECT_PROCESS;
714     return 0;
715 }
716 return -1;
717 }
718
719 static void ensure_contract(void)
720 {
721     struct agent_execution_contract_control control;
722     struct agent_execution_contract_result result;
723
724     if (harness_contract_ready)
725         return;
726     memset(harness_nodes, 0, sizeof(harness_nodes));
727     for (uint index = 0; index < HARNESS_CONTRACT_NODES; index++) {
728         struct agent_execution_contract_node *node = &harness_nodes[index];
729         uint64 capabilities = 0;
730         uint64 accepted = 0;
731         uint64 output = 0;

```

目标 Agent 经 claim 取得描述符后，内核先核对生命周期、owner 身份、通道代次、请求号、槽位代次和目标身份，再验证父权限、workflow policy 和子配置的 capability 与工具集合包含关系。complete 还会复核领取回执中的 worker、线程 generation 与完整 Task tuple。

代码 14: complete 与领取回执的完整绑定核验

c

```

489 agent_task_delegate_receipt_binding_matches(
490     const struct agent_task_delegate_receipt *receipt,
491     const struct proc *worker, const struct thread *thread,
492     const struct agent_task_delegate_complete *complete,
493     struct workflow_lifecycle_key lifecycle)
494 {
495     return receipt != 0 && receipt->valid && worker != 0 && thread != 0 &&
496         complete != 0 &&
497         workflow_lifecycle_key_equal(receipt->lifecycle, lifecycle) &&
498         receipt->worker_pid == worker->pid &&
499         receipt->worker_agent_id == worker->agent_id &&
500         receipt->worker_control_id == worker->agent_control_id &&
501         receipt->worker_thread_generation == thread->identity_generation &&
502         receipt->owner_pid == complete->owner_pid &&
503         receipt->owner_control_id == complete->owner_control_id &&
504         receipt->channel_generation == complete->channel_generation &&
505         receipt->request_id == complete->request_id &&
506         receipt->slot_generation == complete->slot_generation &&
507         receipt->task_id == complete->task_id &&
508         receipt->correlation_id == complete->correlation_id;

```

这条端到端路径分别观察用户态编码失败、内核 schema/route 拒绝、任务终态和工具副作用失败。执行约定测试还覆盖过期生命周期、过期约定、未知节点、工具不匹配、截止时间 和 依赖终止状态（terminal state）。副作用前检查产生的完成记录关联执行记录票号

与输出来源；直接调用或 V3 调用在关键记录不足时返回 NO_SPACE/RETRY，使拒绝和取消结果能够回溯到上下文与执行记录。

任务通道在提交后维护已接纳和运行状态、生命周期引用以及回调引用；硬截止时间触发过期，完成或拒绝进入统一的完成队列。状态机测试核对生命周期引用完整性，并确认过期任务按照终态规则结算。

4.2.3. 长期 Guest 与 Host Harness

Host Harness 为整个会话启动一个 `agentharness_ucore` Guest。每个 Host Agent 通过 runtime SPAWN 对应一个 Guest 进程，动态 Task 则通过原生 Task Channel 进入同一 lifecycle。Host adapter 核对 spawn identity、task id、目标 Agent 和 terminal 状态，网络与模型协议仍位于 Host 侧，智能体身份、工具权限和任务状态由 Guest 与内核验证。

测试运行还检查关闭、取消和过期后的清理过程。Host 只有在没有已领取 Task 时才请求关闭；Guest 先收敛 Execution Contract 与 Task resource，再退出生命周期。内核利用生命周期代次阻止回收后的迟到 Task completion 落到新的 workflow。

4.2.4. 版本化开发与独立 Guest 运行

Nexus 的开发 broker 先把路径限制为 `user/src/nexus*_ucore.c`，再核对当前内容的 SHA-256 revision。创建文件使用 missing，修改文件使用前一次 receipt 返回的 revision；冲突请求只返回当前 revision，不写入目标。提交过程采用同目录临时文件、fsync、原子替换和目录同步。

代码 15：写入前的路径、revision 与原子提交入口

Python

```

428     def write_file(self, path: object, content: object, expected_revision: object) ->
        DevelopmentResult:
429         try:
430             relative = _validate_program_path(path)
431             body = _decode_utf8(content, "content", MAX_WRITE_BYTES)
432             expected = _validate_revision(expected_revision)
433             target = self._target(relative)
434         except NexusDevelopmentError as error:
435             return self._operation_error(str(error))
436         current = self._revision(target)
437         if current != expected:
438             return self._operation_error("revision_conflict", current)
439         try:
440             previous, revision = self._atomic_replace(target, body)
441         except (OSError, NexusDevelopmentError) as error:
442             code = error.args[0] if isinstance(error, NexusDevelopmentError) else "
atomic_commit_failed"
443             return self._operation_error(str(code), current)

```

构建在会话私有临时工作树中进行，目标必须与源文件同名，工具链固定为 `riscv64-linux-gnu-`。成功结果生成 build id 并保存内核与测试镜像；运行工具只接受当前 broker 保存的成功 build id，每个用例复制一份独立镜像，再以单 Hart、128 MiB 内存和 30 秒上限启动 Guest。串口输入、实际输出、退出状态、日志摘要和超时状态都写入结果 artifact。

代码 16: 运行请求的 build 与用例数量约束

Python

```

1157     def run_ucore_program(
1158         self, build_id: object, cases: object
1159     ) -> DevelopmentResult:
1160         if not isinstance(build_id, str) or not _DIGEST_RE.fullmatch(build_id):
1161             return self._operation_error("build_id_invalid")
1162         record = self._builds.get(build_id)
1163         if record is None:
1164             return self._operation_error("build_id_unknown")
1165         if (
1166             not isinstance(cases, Sequence)
1167             or isinstance(cases, (str, bytes, bytearray))
1168             or not 1 <= len(cases) <= MAX_RUN_CASES
1169         ):
1170             return self._operation_error("run_cases_invalid", record.source_revision)

```

在线 DeepSeek 计算器任务使用同一个 build 分别执行正常表达式、非法字符、空输入、运算符错误和除零测试，5 次输出与退出状态全部匹配。模型自行选择单 Agent 方案，在 5 个模型轮次中调用读取、写入、构建和运行；4 次产品工具调用都形成 native Task，并在长期 Guest 中完成 Artifact 与 Context 交付。开发完成门在全部证据齐备前拒绝最终答案，后续源码修改也会使旧 build 与运行集合失效。

4.2.5. 失效状态与恢复分支

测试主动构造过期代次、错误 schema、资源不足、SQ/CQ 协议失步、实时查询缺口和失效 artifact 句柄。各接口分别返回 STALE、DENIED、NO_SPACE、RETRY 等状态，并保持尚未提交的副作用不可见。批量 Metadata 测试在同一调用中依次提交成功项、非法标志、再次成功、选择器冲突、缺失对象和状态更新，核对逐项状态、完整处理前缀与前项保留；输入输出重叠、输出页准备失败和权限不足会在首项提交前停止。

实时查询测试先安装 Typed Watch，再在一个批次中创建并更新同一对象，确认 ENTER、UPDATE 按 generation 顺序到达，随后删除文件并取得 LEAVE。通用 Watch 接口会拒绝 FILE_QUERY，通用清理也会保留仍在工作的 Typed Watch。队列出现缺口时，订阅者以完整快照和对应 generation 完成重同步；截断快照不会被用于确认恢复。

任务通道在协议失步后要求显式 RESYNC。取消测试由同一活动生命周期内的 controller 通过 complete 接口的取消标志，以及 owner、通道、请求、槽位、任务和关联号的准确组合发起，同时检查 ORCHESTRATE、WAIT_CANCEL 与 TASK route。返回 OK 只确认 control request 已接收；任务已经 CLAIMED 时，provider 仍须清理预绑定结果并确认最新终态 offer。

Catalog 测试与通用 Harness 产品路径都让首次页面装载从空 generation 开始，后续分页和读取携带已经核验的 generation。稳定页面会复用已验证的 Catalog；页面变化时，Guest 先将 control 记录推进到 STALE，清理旧窗口，再从 cursor 0 重建。构建过程中只有 BUILDING 状态可见，全部批次成功并通过 Typed Watch 核对后才发布 READY。在线开发验收的读取路径记录 used_index=1，Catalog 共 6 项、候选 5 项，Host 只处理 Guest 返回的候选；固定角色 Nexus 产品命令已经退役。

5. 总结与展望

5.1. 阶段性结论

本项目在 RISC-V uCore 中实现了内核中的智能体运行机制。AgentOS-uCore 以现有的进程、页表、VFS、系统调用和调度器为基础，把身份、工具、上下文、文件查询、事件等待、资源和调度纳入同一工作流。六组机制让智能体在发生副作用时都具有明确的身份、能力位、访问范围、代次、数据来源和资源账户。

项目取得了三方面进展。第一，Agent 已从应用层标签变为内核能够识别的进程身份，普通进程和 Agent 进程可以在同一个 uCore 中共存，高频 Context 数据也能从固定用户地址直接读取。第二，33 项工具目录、ABI vNext 的 V2/V3、private Context、Context Artifact Store、实时查询、128 字节动态 Task、查询预测、资源账户和 EEVDF 贯通了 Agent 从调用、等待到取得资源服务的主要过程。第三，Nexus 已从固定名称分工扩展为共同 Agent Loop，并把 Host Agent 接入一个长期运行 Guest 的原生 Task Channel：capability、允许工具、额度、提示和任务描述决定行为，版本化写入、隔离构建和独立 Guest 测试使 DeepSeek 能够沿着读取、修改、编译和验证持续迭代。

实验展示了各条路径的性能特征。正式 campaign 在 16 次相互独立的赛题平台启动中设置 $K = 4$ 次发现查询，并使 A/B 与 B/A 各占 8 组。核心窗口的遍历查询和索引复用中位数分别为 121.206 ms 与 19.500 ms，中位耗时之比为 6.216 倍，16/16 个配对均更快，配对中位差为 -105.668 ms；完整流程的中位数由 786.684 ms 降至 697.807 ms，中位耗时之比为 1.127 倍，16/16 个配对同样更快，配对中位差为 -89.782 ms。批量登记 96 条元数据记录并复用 Catalog 后，检查记录数由 385 降至 5（77 倍），两条路径的结果哈希一致。12 组文件目录网格的中位加速比介于 1.164–2.808 倍；504 条 EEVDF 唤醒探针均在 0–1 个 tick 内得到服务。

5.2. 当前技术限制

当前测试环境是 RV64 赛题平台的单 Hart，工作流 EEVDF 的实体迁移和多核竞争尚未测量。实时查询单次最多返回 8 个结果，ABI 还没有分页游标；事件队列、Context 路径、Artifact metadata、任务通道、预测转移表和调度实体表也采用固定容量。在线通用 Harness 已把 Host Agent 与一个长期运行 Guest 的 native Task Channel 接通；当前联合回归覆盖 2 个配置 Agent 和 2 个嵌套 Task，长 Provider 延迟、大规模并行子任务与在线取消仍需扩展。调度观测使用 10 ms 的 tick，适合判断唤醒是否跨越节拍，不足以分辨微秒级差异。

5.3. 后续工作

下一阶段将扩展长期 Guest 会话中的并行任务、取消和 Provider 故障回归，并为版本化写入、隔离构建和独立 Guest 测试补充用户审批策略、更丰富的目标清单与构建缓存。随后补充 Artifact 正文持久化、分页查询和长时间压力场景，重点观察摘要压缩、任务积压、订阅事件密集到达与预取队列的背压行为；Host PREFETCH_HINT 还需要进入在线 workspace

broker 的读取与命中回执链。工作流 EEVDF 将扩展到 SMP 和真实 RISC-V 硬件，分析实体迁移、唤醒和资源记账。

5.4. 比赛收获

比赛过程中，我们对“操作系统应该负责什么”的判断逐渐具体。早期开发把重点放在为 uCore 增加面向智能体的接口；模型请求、Guest 应用、内核对象和 Host 服务贯通以后，职责分工变得清晰：用户态与 Host 保留模型策略，uCore 内核执行层统一处理执行者身份、内核可见的工具副作用、对象代次、任务终态和工作流资源账户。

我们也重新认识了“功能跑通”和“系统闭环”的差别。系统闭环要求模块在取消、超时、进程退出、代次复用和资源耗尽时仍然保持一致。组合测试迫使我们把问题逐项说清：Task Channel 的取消和迟到完成至多产生一条 terminal CQE；Metadata Catalog 与 Typed Watch 必须让旧 generation 的窗口失效；Workflow Fence 和退出路径则要在关闭时回收对象与资源。做过这些检查以后，我们逐渐习惯在跨层接口中先说明谁保存权威状态、谁传递引用、谁负责最后校验，再开始写实现。

调试方法也发生了变化。赛题平台串口、固定 ReplayProvider、分配器故障与重启矩阵，以及 Context、状态码和资源计数，让我们能够把模型输出的随机性与内核、协议和文件系统问题分开：先把失败稳定复现，再沿 Host、Guest、系统调用和内核状态逐层缩小范围。测试会检查真实调用路径、Context 序号、失败后的继续运行，以及块、inode 和工作流资源计数是否回到基线。优化前的配对测量暴露出完整流程没有保留核心收益；定位到逐项元数据登记后，我们用批量登记和 Catalog 复用重组工作负载。新的 16 组均衡配对中，完整流程 16/16 组更快，配对中位差为 -89.782 ms 。这个过程让我们更重视让测量直接指向下一步实现。

协作方面，我们体会最深的是，系统项目不能只依靠口头同步。共享 ABI 的结构大小和字段偏移、可以直接运行的构建命令、失败时能够定位到源码的测试，以及随实现同步更新的设计文档，都是协作接口的一部分。它们让内核、Guest 和 Host 的修改能够被另一侧独立复核，也让后来发现的跨模块问题不必从头猜测。回头看，这次比赛真正改变的是我们的工作顺序：先明确可信主体、职责范围和状态权威，再用能够复现、能够归因的实验检验实现；异常路径、文档和复现命令随后也成为功能交付的一部分。

6. 参考文献

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [2] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. Artificial intelligence risk management framework: Generative artificial intelligence profile. NIST AI 600-1, National Institute of Standards and Technology, Gaithersburg, MD, July 2024. URL <https://doi.org/10.6028/NIST.AI.600-1>.
- [3] OWASP Gen AI Security Project. OWASP Top 10 for LLM Applications 2025. Version 2025, released November 18, 2024, November 2024. URL <https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/>. Licensed under CC BY-SA 4.0.
- [4] OWASP Gen AI Security Project, Agentic Security Initiative. Agentic AI – threats and mitigations. Version 1.1, December 2025. URL <https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/>. Licensed under CC BY-SA 4.0.
- [5] Model Context Protocol. Mcp specification 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28>, 2026.
- [6] A2A Project. A2a protocol specification v1. <https://a2a-protocol.org/latest/specification/>, 2026.
- [7] LearningOS Community. uCore-Tutorial-Code-2025S. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础.
- [8] RISC-V International. The RISC-V instruction set manual, volume i: Unprivileged architecture. <https://docs.riscv.org/reference/isa/v20260120/unpriv/intro.html>, 2026.
- [9] RISC-V International. The RISC-V instruction set manual, volume ii: Privileged architecture. <https://docs.riscv.org/reference/isa/v20260120/priv/priv-intro.html>, 2026.
- [10] Ion Stoica and Hussein Abdel-Wahab. Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation. Technical Report TR-95-22, Department of Computer Science, Old Dominion University, November 1995. URL <https://people.eecs.berkeley.edu/~istoica/papers/eevdf-tr-95.pdf>.
- [11] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026.